

Capítulo 13

Teste

Este capítulo discute os *fundamentos* (Seção 13.1) da área de teste de software. Na sequência são apresentados os *níveis de teste de funcionalidade* (Seção 13.2) e os *testes suplementares* (Seção 13.3) mais frequentemente usados. Depois, são apresentadas técnicas específicas de teste, iniciando pelo *teste estrutural* (Seção 13.4), seguido da técnica de *teste funcional* (Seção 13.5). É apresentada a técnica de *desenvolvimento dirigido por testes (TDD)* (Seção 13.6), e também são discutidas métricas (Seção 13.7) a serem usadas nas atividades de teste e as atividades de depuração de programas (Seção 13.8).

Por melhores que sejam as técnicas de modelagem e especificação de software, por mais disciplinada e experiente que seja a equipe de desenvolvimento, sempre haverá um fator que faz que o teste de software seja necessário: o *erro humano*. É um mito pensar que bons desenvolvedores, bem concentrados e com boas ferramentas serão capazes de desenvolver software sem erros (Beizer, 1990).

A Lei de Murphy (Bloch, 1977), em vários de seus enunciados, parece falar diretamente para a indústria de software:

- "Se alguma coisa pode sair errado, sairá (no pior momento possível)".
- "Se tudo parece estar indo bem é porque você não olhou direito".
- "A natureza sempre está a favor da falha oculta".
- "O defeito sempre está no último lugar em que você procura".

Durante muitos anos, a tarefa de teste de software foi considerada um castigo para os programadores. O teste era considerado uma tarefa ingrata, porque se esperava justamente que os desenvolvedores construíssem software de boa qualidade. A necessidade de testes destacava essa incapacidade que era indesejada.

A área de teste podia ser caracterizada, então, por situações caricatas como as seguintes:

- "Depois eu testo."
- "Na minha máquina funcionou..."
- "Vamos deixar os testes para a próxima fase."
- "Temos que entregar o produto na semana que vem" etc.

Entretanto, as coisas mudaram. Conforme visto em alguns modelos de ciclo de vida, a disciplina de teste passou a ser considerada extremamente importante. Hoje ela é parte do processo de desenvolvimento de software. Os métodos ágeis também incorporaram o teste de software como uma atividade crítica, assumindo inclusive que os casos de teste deveriam passar a ser escritos antes das unidades de software que pretendem testar.

Além disso, grandes empresas desenvolvedoras de software passaram a contratar o teste de software de forma independente (fábricas de teste). Ou seja, os responsáveis pelo teste não são mais apenas os desenvolvedores, mas equipes especialmente preparadas para executar essa tarefa.

13.1 Fundamentos

A tarefa de testar software, porém, não é simples. Em algumas situações, pode ser mais difícil

elaborar bons casos de teste do que produzir o próprio software. Assim, muita sistematização e controle são necessários para que a atividade de teste de software deixe de ser uma tarefa totalmente *ad hoc* e ingênua para se tornar uma atividade de engenharia com resultados efetivos e previsíveis.

Esta seção apresenta os fundamentos da área de teste de software, iniciando com conceitos básicos, seguida de uma apresentação dos diferentes níveis e objetivos dos testes, bem como das principais técnicas de teste que podem ser imediatamente aplicadas, seja manualmente, seja pelo uso de ferramentas automatizadas.

13.1.1 ERRO, DEFEITO E FALHA

Inicialmente, convém definir alguns termos que, caso contrário, poderiam ser considerados sinônimos, mas que na literatura de teste têm significados bastante precisos:

- Um *erro* (*error*) é uma diferença detectada entre o resultado de uma computação e o resultado correto ou esperado.
- Um *defeito* (*fault*) é uma linha de código, bloco ou conjunto de dados incorretos que provocam um erro observado.
- Uma *falha* (*failure*) é um não funcionamento do software, possivelmente provocada por um defeito, mas com outras causas possíveis.
- Um *engano* (*mistake*), ou *erro humano*, é a ação que produz ou produziu um defeito no software.

Cabe observar aqui que o termo *fault* (defeito) algumas vezes é traduzido como *falha*, mas a falha em si (*failure*) é a observação de que o software não funciona adequadamente. Existem falhas que são provocadas por defeitos no software, mas outras que são provocadas por dados incorretos ou problemas tecnológicos (como falha de leitura, segurança ou comunicação).

Assim, nem todas as falhas são provocadas por defeitos. A área de teste de software ocupa-se principalmente das falhas provocadas por defeitos para que os defeitos sejam corrigidos e, assim, essas falhas nunca mais ocorram.

Já as falhas provocadas por causas externas ao software costumam ser assunto da área de *tolerância a falhas* (Linden, 1976). Os requisitos suplementares de tolerância a falhas de um sistema é que vão estabelecer, por exemplo, como o software se comporta no caso de uma falha de comunicação, armazenamento, segurança etc. Note que o requisito vai especificar o comportamento do software em uma situação de falha provocada externamente. Assim, caso o software se comporte de acordo com sua especificação e a falha ocorra, pode-se dizer que ela não é provocada por um defeito no software e que, pelo menos em relação a essa situação, ele *parece estar* livre de defeito.

No parágrafo anterior evitou-se afirmar categoricamente que o software está livre de defeito, porque, de fato, tal afirmação dificilmente poderá ser feita para sistemas que não sejam triviais. A maioria dos sistemas de médio e grande porte pode conter defeitos ocultos, porque a quantidade de testes necessária para garantir que estejam livres de defeitos pode ser impraticável, a não ser que estes sistemas já tivessem sido definidos desde o seu início com boas técnicas de teste automatizado.

Assim, a área de teste de software ocupa-se em definir conjuntos finitos e exequíveis de teste que, mesmo não *garantindo* que o software esteja livre de defeitos, consigam localizar os mais prováveis, permitindo assim sua eliminação.

Uma máxima da área de teste de software afirma que o teste não consegue provar que o software está livre de defeitos. Ele apenas consegue provar que o software *possui* algum defeito.

13.1.2 VERIFICAÇÃO, VALIDAÇÃO E TESTE

Outra distinção que convém ser feita é entre verificação, validação e teste:

- *Verificação*: consiste em analisar o software para ver se ele está sendo construído de acordo com o que foi especificado.
- *Validação*: consiste em analisar o software construído para ver se ele atende às verdadeiras necessidades dos interessados.
- *Teste*: é uma atividade que permite realizar a verificação e a validação do software.

Assim, a pergunta-chave para a *validação* é “Estamos fazendo a coisa certa?”, enquanto a pergunta-chave para a *verificação* é “Estamos fazendo a coisa do jeito certo?”.

No caso da validação, trata-se de saber se os requisitos incorporados ao software refletem efetivamente as necessidades dos interessados (cliente, usuário etc.). Isso normalmente é o objetivo dos testes em nível de aceitação ou homologação.

Já no caso da *verificação*, trata-se de saber se o produto criado atende aos requisitos que foram especificados da forma mais adequada possível, ou seja, se está livre de defeitos e possui outras características de qualidade já definidas. Isso, normalmente é o objetivo dos testes em nível de unidade, integração e sistema.

13.1.3 TESTE E DEPURAÇÃO

Enquanto a atividade de teste consiste em executar sistematicamente o software para encontrar erros desconhecidos, a *depuração* é a atividade que consiste em buscar a causa do erro, ou seja, o defeito oculto que a está causando.

O fato de se saber que o software não funciona não significa que necessariamente se saiba qual é (ou quais são) a(s) linha(s) de código que provoca(m) esse erro. A depuração pode ser uma atividade dispendiosa. Por isso, os processos mais modernos recomendam que a integração dos sistemas seja feita de forma incremental e que sejam integradas pequenas partes de código de cada vez, pois, se um sistema funcionava antes da integração e passou a falhar depois dela, o defeito provavelmente está nos componentes que acabaram de ser integrados, e não nos componentes que já funcionavam.

13.1.4 STUBS E DRIVERS

Frequentemente, partes do software precisam ser testadas de modo isolado, mas em geral elas se comunicam com outras partes.

Quando um componente *A* que vai ser testado chama operações de outro componente *B* que ainda não foi implementado, pode-se criar uma implementação simplificada de *B*, chamada *stub*, que será utilizada no lugar de *B*. Por exemplo, essa implementação simplificada pode ser uma função que, em vez de realizar um cálculo, retorna um valor predeterminado, mas que será útil para testar o componente *A*.

Suponha que *A* é uma classe que precisa usar um gerador de números primos *B*. A implementação completa de *B* capaz de gerar o *n*-ésimo número primo através da função pode ainda não ter sido feita. Mas, possivelmente, para testar *A* não será necessário gerar mais do que alguns poucos números primos, por exemplo, os cinco primeiros. Então *B* poderia ser substituído por uma função *stub* que gerasse apenas os cinco primeiros números primos, implementada da seguinte forma (código em Python):

```
class B:
```

```
def primo(self, n): #stub
    if n == 1: return 2
    elif n == 2: return 3
    elif n == 3: return 5
    elif n == 4: return 7
    elif n == 5: return 11
    else: return None

b = B()
assert b.primo(1) == 2
assert b.primo(2) == 3
assert b.primo(3) == 5
assert b.primo(4) == 7
assert b.primo(5) == 11
```

Assim, o *stub* é implementado com uma simples estrutura de seleção que é definida apenas para os valores de n variando de 1 a 5. Os comandos "assert" estão ali para demonstrar que a função realmente retorna os valores corretos dentro do intervalo definido. Dessa forma, a classe A pode ser testada sem que B tenha sido efetivamente implementada. A [Figura 13.1](#) representa esquematicamente a relação entre a classe a ser testada e o *stub* da classe dependente dela.

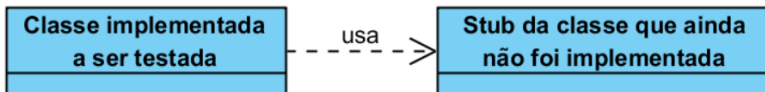


Figura 13.1 *Stub*

Por outro lado, muitas vezes é o módulo B que já está implementado, mas o módulo A que chama as funções de B ainda não foi implementado. Nesse caso, deverá ser implementada uma simulação do módulo A , denominada *driver*. Essa simulação deverá chamar as funções do módulo B e, de preferência, executar todos os casos de teste necessários para testar B , de acordo com a técnica de teste adotada. A [Figura 13.2](#) mostra esquematicamente a relação entre o *driver* e a classe a ser testada.

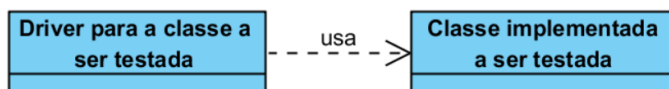


Figura 13.2 *Driver*

A diferença entre o *stub* e o *driver* reside na direção da relação de dependência entre os componentes. Se o componente testado depende (ou seja, chama operações) do componente simulado, então o componente simulado é um *stub*. Inversamente, se o componente simulado é que chama as operações do componente testado, então o componente simulado é um *driver*. Assim, um *stub* e um *driver* são implementações simplificadas ou simuladas de componentes de sistema, construídas especificamente para atividades de teste.

Stubs são, normalmente, um código do tipo *throw-away*, porque depois da implementação real da classe que eles representam, eles não são mais necessários.

Por outro lado, os *drivers* são patrimônio de teste. Eles devem ser guardados para a execução

de futuros testes de regressão (Seção 13.2.6). Isso porque os *drivers* terão sido construídos de acordo com técnicas especiais para testar sistematicamente as combinações de entradas mais importantes para as classes ou módulos aos quais eles estão associados, conforme será visto mais adiante neste capítulo. As seções 13.5 e 13.6 apresentam vários exemplos de *drivers*.

13.1.5 Fixtures

Fixtures são variáveis ou objetos criados especificamente para serem usados nas atividades de teste automatizado. Em geral, cada teste será executado em três etapas:

1. *Preparar o cenário*. É o momento em que as *fixtures* são criadas.
2. *Executar o teste*. Consiste na chamada das funções que vão ser testadas passando as *fixtures* adequadas em cada caso e verificando se o resultado é o esperado.
3. *Limpar o ambiente*. Todas as *fixtures* devem ser destruídas após o teste.

A criação de *fixtures* é uma atividade que frequentemente gera código repetido, pois em muitos casos as mesmas *fixtures* são usadas em mais de um teste. Pesquisas ainda são feitas hoje para identificar formas de minimizar este problema (Silva & Vilain, 2016).

13.2 Níveis de Teste de Funcionalidade

Os objetivos dos testes podem variar bastante e abrangem desde verificar se as funções mais básicas do software estão bem implementadas até validar os requisitos junto ao cliente. A classificação de testes mais empregada considera que existem testes de funcionalidade (unidade, integração, sistema, ciclo de negócio, aceitação, operação etc.), que serão detalhados nesta seção, e testes suplementares (*performance*, segurança, tolerância a falhas etc.). Deve-se tomar cuidado para não confundir teste de funcionalidade (um nível de teste) com teste funcional (uma técnica de teste); o primeiro termo se refere ao objetivo do teste e o segundo à forma como ele é executado. A técnica de teste funcional é apresentada na Seção 13.5.

Os testes de funcionalidade têm como objetivo basicamente verificar e validar se as funções implementadas estão corretas nos seus diversos níveis.

13.2.1 TESTE DE UNIDADE

Os testes de unidade são os mais básicos e costumam consistir em verificar se um componente individual do software (*unidade*) foi implementado corretamente. Esse componente pode ser um método ou procedimento, uma classe completa ou, ainda, um pacote de funções ou classes de tamanho pequeno a moderado. Em geral, essa unidade está isolada do sistema do qual faz parte.

Os testes de unidade costumam ser realizados pelo próprio programador, e não pela equipe de teste. A técnica de desenvolvimento orientado a testes, ou TDD (Beck, 2003), recomenda inclusive que, antes de o programador desenvolver uma unidade de software, ele deve desenvolver o seu *driver* de teste, que, conforme foi visto, é um programa que obtém ou gera um conjunto de *fixtures* que serão usadas para testar sistematicamente o componente que ainda vai ser desenvolvido.

O teste de unidade pode se valer bem da técnica de teste estrutural (Seção 13.4), que vai garantir pelo menos que todos os comandos e decisões do componente implementado tenham sido exercitados para verificar se eles têm defeitos. Mas quando se adota TDD é necessário iniciar, pelo menos, com o teste funcional (Seção 13.5), que verifica o componente relativo à sua especificação. Usualmente, testes estruturais são realizados apenas em código legado ou suspeito, ou ainda código no qual não se tenha muita confiança de que os testes funcionais tenham tido suficiente cobertura de código. As seções 13.4 e 13.5 apresentam vários exemplos de testes de unidade usando as técnicas estrutural e funcional.

13.2.2 TESTE DE INTEGRAÇÃO

Testes de integração são feitos quando unidades (classes, por exemplo) estão prontas, são testadas isoladamente e precisam ser integradas em um *build* para gerar uma nova versão de um sistema. Entre as estratégias de integração, em geral são citadas as seguintes:

- *Integração big-bang*: consiste em construir as diferentes classes ou componentes separadamente e depois integrar tudo junto no final. É uma técnica não incremental, utilizada no ciclo de vida Cascata com Subprojetos. Tem como vantagens o alto grau de paralelismo que se pode obter durante o desenvolvimento e o fato de não precisar de *drivers* e *stubs* durante a integração (embora eles sejam necessários nos testes de unidade), mas como desvantagem cita-se o fato de não ser incremental (portanto, inadequada para o Processo Unificado e métodos ágeis). Além disso, a integração de muitos componentes ao mesmo tempo pode dificultar bastante a localização dos defeitos, pois eles poderão estar em qualquer um dos componentes.
- *Integração bottom-up*: consiste em integrar inicialmente os módulos de mais baixo nível, ou seja, aqueles que não dependem de nenhum outro, e depois ir integrando os módulos de nível imediatamente mais alto. Assim, um módulo só é integrado quando todos os módulos dos quais ele depende já foram integrados e testados. Dessa forma, não é necessário escrever *stubs*, mas em compensação as funcionalidades de mais alto nível do sistema serão testadas tardiamente, quando os módulos de nível superior forem finalmente integrados.
- *Integração top-down*: consiste em integrar inicialmente os módulos de nível mais alto, deixando os mais básicos para o fim. A vantagem está em verificar inicialmente os comportamentos mais importantes do sistema em que repousam as maiores decisões. No entanto, como desvantagem há o fato de que muitos *stubs* são necessários. O teste, para ser efetivo, precisa de bons *stubs*, caso contrário, na integração dos módulos de nível mais baixo poderão ocorrer problemas inesperados.
- *Integração sanduíche*: consiste em integrar os módulos de nível mais alto da forma *top-down* e os de nível mais baixo da forma *bottom-up*. Essa técnica reduz um pouco os problemas das duas estratégias anteriores, mas seu planejamento é mais complexo.
- *Integração incremental*: consiste em integrar módulos à medida em que vão ficando prontos, independentemente de serem de nível mais alto, mais baixo ou até intermediário. O planejamento do desenvolvimento dos módulos é que vai determinar se a integração ocorrerá com características mais *top-down*, *bottom-up* ou *sandwich*.

A principal desvantagem da maioria das técnicas (exceto a *big-bang*) é o fato de que as classes ou componentes individuais que vão ser testados na integração necessitam comunicar-se com outros componentes ou classes que ainda não foram testados ou sequer escritos. Nesse caso, as interfaces que ainda não existem são supridas pelos *stubs*, que são implementações incompletas e simplistas e que se tornam descartáveis depois que o código real é produzido.

O problema com *stubs*, portanto, é que se perde tempo desenvolvendo software que não vai ser efetivamente entregue. Além disso, nem sempre é possível saber se a simulação produzida pelo *stub* será suficientemente adequada para os testes.

No caso do desenvolvimento iterativo, especialmente no caso dos métodos ágeis, nem sempre se sabe *a priori* em que ordem os componentes serão integrados, porque vários desenvolvedores estarão trabalhando em paralelo em componentes de diferentes níveis da hierarquia de dependência. Assim, a cada integração deverá ser avaliado se o componente já possui no sistema os componentes dos quais ele depende e os componentes que dependem dele. Na falta destes, usam-se

drivers ou *stubs*, como nos testes de unidade.

É necessário ainda estar atento às versões dos componentes do sistema, de forma que a integração sempre deixe claro quais versões de quais componentes foram efetivamente testadas juntas (**Capítulo 10**). Assim, os testes de integração nada mais serão do que os testes de unidade executados no contexto da aplicação completa e não de forma isolada. Um bom sistema de controle de versões poderá identificar quais testes devem ser realizados sempre que um novo componente for integrado ao *build*. Para isso, ele deverá usar as relações de dependência entre itens de configuração, conforme explicado na **Seção 10.1.2**.

13.2.3 TESTE DE SISTEMA

O *teste de sistema* visa verificar se a versão corrente do sistema permite executar processos ou casos de uso completos do ponto de vista do usuário que executa uma série de operações de sistema em uma interface e é capaz de obter os resultados esperados.

Assim, o teste de sistema pode ser encarado como o teste de execução dos fluxos de um caso de uso expandido. Se cada uma das operações de sistema (passos do caso de uso) já estiver testada e integrada corretamente, então deve-se verificar se o fluxo principal do caso de uso pode ser executado corretamente, obtendo os resultados desejados, bem como os fluxos alternativos (**Wazlawick, 2015**).

Normalmente, o teste de sistema é realizado com o uso da técnica funcional, ou seja, não se examina a estrutura interna do código, apenas a forma como o sistema se comporta em relação a sua especificação.

Considera-se que o teste de sistema somente é executado em uma versão (*build*) do sistema em que todas as unidades e os componentes integrados já foram testados. Caso muitos erros sejam encontrados durante o teste de sistema, o usual é abortar o processo e refazer, ou mesmo replanejar, os testes de unidade e integração, para que uma versão suficientemente estável do sistema seja produzida e possa ser testada em nível de sistema. Um exemplo completo de teste de sistema é apresentado na **Seção 13.5.3**.

13.2.4 TESTE DE ACEITAÇÃO

O *teste de aceitação* ou *homologação* costuma ser realizado utilizando-se a interface final do sistema. Ele pode ser planejado e executado exatamente como o teste de sistema, mas a diferença é que é realizado pelo usuário final ou cliente, e não pela equipe de desenvolvimento.

Em outras palavras, enquanto o teste de sistema faz a *verificação* do sistema, o teste de aceitação faz sua *validação*. O teste de aceitação tem como objetivo principal, portanto, a validação dos requisitos do software, e não apenas a verificação de defeitos em relação às especificações. Ao final do teste de aceitação, o cliente poderá aprovar a versão do sistema testada ou solicitar modificações.

O teste de aceitação pode ter mais duas variantes:

- *Teste alfa*: esse teste é efetuado pelo cliente ou seu representante de forma livre, sem o planejamento e a formalidade do teste de sistema. O usuário vai utilizar o sistema e suas funções livremente e, por isso, esse teste também é chamado de *teste de aceitação informal*, em oposição ao *teste de aceitação formal*, que deveria seguir o mesmo planejamento utilizado pelo teste de sistema.
- *Teste beta*: esse teste é ainda menos controlado pela equipe de desenvolvimento. Nele, versões operacionais do software são disponibilizadas para vários usuários, que, sem acompanhamento direto nem controle da empresa desenvolvedora, vão explorar o sistema e suas

funcionalidades. Normalmente, versões beta de sistemas expiram após um período predeterminado, quando então os usuários são convidados a fazer uma avaliação do sistema. Os sistemas feitos sob medida (*tailored*) devem preferencialmente ser testados pelo teste de aceitação formal, pois após o teste haverá uma aceitação formal de que o sistema está correto. Já os sistemas de prateleira (*off-the-shelf*) são normalmente testados por testes de aceitação alfa e beta.

13.2.5 TESTE DE CICLO DE NEGÓCIO

O teste de ciclo de negócio é uma abordagem possível tanto no teste de sistema quanto no teste de aceitação formal, e consiste em testar uma sequência de casos de uso que correspondem a um possível ciclo de negócio da empresa.

Assim, em vez de testar os casos de uso isoladamente, o analista ou cliente vai testá-los no contexto de um ciclo de negócio. Pode-se, por exemplo, testar a sequência de compra de um item do fornecedor, seu registro de chegada em estoque, sua venda, entrega e respectivo pagamento. São vários casos de uso relacionados no tempo, pois representam o ciclo de vida em um item em relação à empresa.

Os testes de ciclo de negócio podem ser planejados a partir de especificações de modelo de negócio (se existirem). Essas especificações costumam ser feitas com diagramas de atividade UML (Wazlawick, 2015) ou BPMN (*Business Process Modeling and Notation*).

13.2.6 TESTE DE REGRESSÃO

O teste de regressão é executado sempre que um sistema em operação sofre manutenção. O problema é que a correção de um defeito no software, ou a modificação de alguma de suas funções, pode ter gerado novos defeitos. Nesse caso, devem ser executados novamente todos os testes de integração e/ou testes de sistema sobre as partes afetadas.

No caso de manutenção adaptativa (Seção 14.2.2), pode ser necessário executar testes de aceitação, com participação do cliente. Mas, no caso da manutenção corretiva (Seção 14.2.1), tais testes não são necessários, visto que a funcionalidade do software, já aprovada pelo cliente, não muda.

O teste de regressão tem esse nome porque, se ao se aplicarem testes a uma nova versão na qual versões anteriores passaram e ela não passar, então considera-se que o sistema *regrediu*.

Como é um teste relativamente demorado e aplicado diversas vezes ao longo do tempo de vida do software, o ideal é que o teste de regressão seja sempre automatizado (Dustin, 2002).

13.2.7 TESTE DE OPERAÇÃO

O teste de operação pode ser uma variante do teste de sistema, aceitação, ciclo de negócio ou regressão que é executado no ambiente final de implantação do sistema. Esse tipo de teste é muito importante, especialmente em sistemas críticos, pois problemas de compatibilidade ou convivência entre sistemas as vezes podem gerar erros completamente inesperados.

Imagine, por exemplo, um sistema que passou em todos os testes no ambiente de desenvolvimento, inclusive pelos testes de aceitação com a presença do cliente. Agora esse sistema será instalado em uma máquina específica nas dependências da organização cliente. Ele é carregado com dados reais de sistemas legados e colocado para funcionar. Em poucas horas, percebe-se que o sistema começa a travar e rapidamente se torna não operacional. O que aconteceu? É a famosa síndrome de "na minha máquina funcionou". Ocorre que a máquina na empresa cliente poderá ter uma versão ligeiramente diferente do sistema operacional, diferente capacidade de memória, processamento ou periféricos, ou ainda, poderá ter instalados na mesma máquina outros produtos de software como antivírus e *firewall*, que podem comprometer de forma inesperada o funcionamento

da aplicação que estava homologada no ambiente de desenvolvimento.

Por outro lado, quando se trata de um sistema a ser instalado em múltiplas máquinas, como por exemplo, sistemas de prateleira. É imprescindível que sejam realizados os testes beta, que neste caso também funcionam como teste de operação.

13.3 Testes Suplementares

Ao contrário dos testes de funcionalidade, que verificam e validam as funções do sistema, ou seja, os processos que podem ser efetivamente realizados, os *testes suplementares* verificam características normalmente associadas aos requisitos suplementares, como *performance*, interface, tolerância a falhas etc. Uma lista não exaustiva de testes suplementares, incluindo apenas os mais comumente realizados, é apresentada nas subseções seguintes.

13.3.1 TESTE DE INTERFACE COM USUÁRIO

O *teste de interface com usuário* tem como objetivo verificar se a interface permite realizar, com eficácia e eficiência, as atividades previstas nos casos de uso. Mesmo que as funções estejam corretamente implementadas, o que já teria sido verificado no teste de sistema, isso não quer dizer que a interface também esteja. Então, em geral, é necessário testar a interface de forma objetiva e específica.

O teste de interface com usuário pode ainda verificar a conformidade das interfaces com normas ergonômicas que se apliquem, como a NBR ISO 9241-11:2018, por exemplo.

No Processo Unificado e outros processos baseados em casos de uso, a base para a realização de testes de interface consistirá também nos casos de uso, de forma análoga ao que acontece no teste de sistema. Porém, agora, em vez de simplesmente verificar se o sistema permite a execução das funcionalidades e a obtenção dos resultados corretos, o teste deverá verificar se a interface está organizada da melhor forma possível para atender ao usuário com eficiência.

Outra vantagem do emprego de casos de uso como base para o teste de interface com usuário reside no fato de que eles apresentam a estrutura de fluxo principal de uma interação do usuário com o sistema. Assim, a interface projetada deverá procurar otimizar a sequência de ações do usuário nesse fluxo principal. Ao mesmo tempo, as ações necessárias para realizar os fluxos alternativos deverão estar disponíveis no momento em que forem necessárias, ou seja, quando o usuário estiver executando os passos do caso de uso que poderiam provocar esta ou aquela exceção.

Em outras palavras, a sequência de operações do caso de uso deve aparecer claramente na interface em uma ordem lógica, evitando-se, por exemplo, que o usuário precise focar sua atenção em diferentes áreas da interface para seguir uma simples sequência lógica. Algumas interfaces exigem que o usuário faça muitos movimentos de braço para levar o *mouse* para cima e para baixo, para a direita e para a esquerda, enquanto executa uma atividade sequencial, o que deveria ser evitado.

Da mesma forma, se ocorrerem exceções que precisarem ser tratadas pelo usuário, a mensagem deverá aparecer em local visível e próximo do foco de atenção desse usuário (por vezes, sistemas colocam mensagens de erro em locais que não são facilmente vistos por ele), assim como os controles de interface necessários para iniciar as ações corretivas também deverão estar próximos à mensagem e ao foco de atenção do usuário sempre que possível (e não três ou quatro páginas adiante, que precisem ser roladas).

13.3.2 TESTE DE PERFORMANCE (CARGA, ESTRESSE E RESISTÊNCIA)

Com o sistema tendo ou não requisitos de desempenho, o *teste de performance* pode ser

importante, especialmente nas operações que serão realizadas com grande frequência ou de forma iterativa. O teste consiste em executar a operação e mensurar seu tempo, avaliando se está dentro dos padrões definidos. O teste de performance também pode ser usado para avaliar a confiabilidade e a estabilidade de um sistema. Existem, assim, pelo menos três tipos de teste de performance:

- *Teste de carga*: é a forma mais simples de teste de performance. Normalmente, o teste de carga é feito para determinada quantidade de dados ou transações que deveriam ser típicos para um sistema e avalia o comportamento do sistema em termos de tempo para esses dados ou transações. Dessa forma, pode-se verificar se o sistema atende aos requisitos de performance estabelecidos e também se existem gargalos de performance para serem tratados.
- *Teste de estresse*: é um caso extremo de teste de carga. Procura-se levar o sistema acima do limite máximo de funcionamento esperado para verificar como ele se comporta. Esse tipo de teste é feito para verificar se o sistema é suficientemente robusto em situações anormais de carga de trabalho. Ele também ajuda a verificar quais seriam os problemas encontrados caso a carga do sistema ficasse acima do limite máximo estabelecido.
- *Teste de resistência*: é feito para verificar se o sistema consegue manter suas características de performance durante um longo período de tempo com uma carga nominal de trabalho. Os testes de resistência devem verificar, basicamente, o uso da memória ao longo do tempo para garantir que não existam perdas acumulativas de memória em função de lixo não recolhido (vazamento de memória), e também se não existe degradação de performance após um substancial período de tempo em que o sistema opera com carga nominal.

Pode-se verificar também, no caso de sistemas concorrentes, o que acontece quando um número de usuários próximo ou acima do máximo gerenciável tenta utilizar o sistema. Algumas vezes, efeitos colaterais indesejados podem surgir nessas situações, por isso esses testes são considerados altamente desejáveis.

13.3.3 TESTE DE SEGURANÇA

Os *testes de segurança* costumam ser considerados parte da área de *segurança computacional*.

Basicamente, os seis tipos de segurança devem ser testados quando os requisitos de sistema assim o exigirem:

- *Integridade de informação*: é uma forma de garantir ao receptor que a informação que ele recebeu é correta e completa.
- *Autenticação*: é a garantia de que um usuário realmente é quem ele diz ser e que os documentos, programas e sites realmente são os esperados.
- *Autorização*: é o processo realizado para verificar se alguma pessoa ou sistema pode ou não acessar determinada informação ou sistema.
- *Confidencialidade*: é a garantia de que pessoas que não têm direito à informação não poderão obtê-la.
- *Disponibilidade*: é a garantia de que pessoas que têm direito à informação conseguirão obtê-la quando necessário.
- *Não repúdio*: é uma forma de garantir que o emissor ou o receptor de uma mensagem não possam alegar, posteriormente, não ter enviado ou não ter recebido a mensagem.

Pode-se dizer que os testes de confidencialidade e disponibilidade se complementam mutuamente, já que se apenas a confidencialidade fosse necessária, sabe-se que a melhor forma de impedir que pessoas não autorizadas tenham acesso à informação é destruindo a informação. Porém, se isso for feito, não haverá mais disponibilidade para as pessoas que têm direito à informação poderem

acessá-la.

13.3.4 TESTE DE RECUPERAÇÃO DE FALHA

Quando um sistema tem requisitos suplementares referentes à tolerância ou à recuperação de falhas, eles devem ser testados separadamente. Basicamente, busca-se verificar se o sistema de fato atende aos requisitos especificados relacionados a essa questão.

Normalmente, o *teste de recuperação de falha* trata de situações referentes a:

- Queda de energia no cliente ou no servidor.
- Discos corrompidos.
- Problemas de comunicação.
- Quaisquer outras condições que possam provocar a terminação anormal do programa ou a interrupção temporária de seu funcionamento.

Assim, o teste de recuperação de falha, também chamado de *teste de robustez*, basicamente consiste em verificar a capacidade de o sistema se tornar novamente operacional após a ocorrência de alguma falha. Pode haver a exigência, na forma de requisito, de que o sistema faça isso automaticamente, mas também é possível que um sistema que precisa ser reativado manualmente seja submetido ao teste de falha. Nos dois casos, pode-se avaliar o tempo necessário para colocar o sistema novamente em operação, se ocorre algum tipo de corrupção nos dados, quantas ações do usuário ocorridas antes da falha são perdidas e devem ser novamente executadas etc.

13.3.5 TESTE DE INSTALAÇÃO

No *teste de instalação*, basicamente, busca-se verificar se o software não entra em conflito com outros sistemas eventualmente instalados em uma máquina, bem como se todas as informações e produtos para instalação estão disponíveis para os usuários instaladores.

O teste de instalação também é associado com o teste de compatibilidade, através do qual se busca verificar se o sistema é compatível com diferentes sistemas operacionais, fabricantes de máquinas, *browsers* etc.

13.4 Teste Estrutural

Em relação às técnicas de teste, existem duas grandes famílias:

- *Testes estruturais*: testes executados com conhecimento do código implementado, ou seja, que testam a estrutura do programa em si (apresentados nesta seção).
- *Testes funcionais* (Seção 13.5): testes executados sobre as entradas e saídas do programa sem que se tenha necessariamente conhecimento do seu código-fonte.

Essas duas famílias de teste incluem várias técnicas, algumas das quais são definidas a seguir. Este capítulo apresenta as principais entre estas técnicas e exemplos completos de casos de teste elaborados com elas. Para um maior aprofundamento em relação à área de teste de software pode-se ainda consultar o livro de Delamaro, Maldonado e Jino (2007).

Os primeiros testes aos quais um sistema é normalmente submetido são os testes de unidade, que têm como objetivo verificar se as funções mais simples do sistema estão corretamente implementadas. Em geral, esse tipo de teste pode ser conduzido como um teste do tipo estrutural, pois o que se deseja é analisar exhaustivamente a estrutura interna do código implementado.

Um dos problemas com os testes de programas é que é impossível definir um procedimento algorítmico que certifique que um programa qualquer está livre de defeitos. Esse problema é computacionalmente *indecidível* (Lucchesi, Simon, Simon & Kowaltowski, 1979). Assim, de certa forma, os testes precisam ser feitos por amostragem.

Wazlawick, R. S. *Engenharia de Software: Conceitos e práticas*. 2ª. ed. Elsevier, 2019.

Então, o teste estrutural não vai testar *todas* as possibilidades de funcionamento de um programa, mas as mais representativas. Em especial, o que se busca com o teste estrutural é exercitar todos os comandos do programa e todas as condições pelo menos uma vez para cada um de seus valores possíveis (verdadeiro ou falso).

Uma primeira técnica de teste estrutural pode ser definida assim: cada comando de seleção ou repetição deve ser testado em pelo menos duas situações: quando a condição de seleção ou repetição é *verdadeira* e quando a condição é *falsa*. No caso dos comandos de repetição deve-se testar como o programa se comporta quando a repetição não é realizada nenhuma vez (uma frequente fonte de defeitos em código) e quando ela é realizada pelo menos uma vez.

Assim, o teste estrutural é capaz de detectar uma quantidade substancial de possíveis erros pela garantia de ter executado pelo menos uma vez todos os comandos e condições do programa. As subseções seguintes apresentam mais detalhes sobre esta técnica.

13.4.1 COMPLEXIDADE CICLOMÁTICA

Uma medida de complexidade de programa bastante utilizada é a *complexidade ciclomática* (McCabe, 1972), a qual pode ser definida, de forma simplificada, assim: se n é o número de estruturas de seleção e repetição no programa, então a complexidade ciclomática do programa é $n+1$. Assim, a complexidade ciclomática mínima é 1, e ela é atribuída a qualquer programa que tenha apenas comandos sequenciais, sem nenhuma decisão ou repetição. Porém, quando estes comandos existem, a quantidade deles é somada ao "1" atribuído ao programa. Por exemplo, um programa com dois IF terá complexidade ciclomática igual a 3. Então, além do valor mínimo 1 inicialmente atribuído ao programa, deve-se *adicionar*:

- Estruturas de seleção no estilo IF: 1 ponto cada.
- Estruturas de seleção no estilo IF-ELSE: 1 ponto cada. Observa-se assim, que o comando "ELSE" não conta pois ele em si não é um comando de decisão.
- Estruturas de seleção no estilo CASE, SWITCH, ELSIF ou ELIF: 1 ponto para cada opção do CASE ou SWITCH, ou para cada ELSEIF ou ELIF. Observe que o comando CASE ou SWITCH isoladamente não contam, mas apenas as opções definidas por eles. Por outro lado, quando se usa ELSEIF ou ELIF, deve-se contar o IF inicial e mais um ponto para cada ELSEIF ou ELIF que o seguir. Assim como o ELSE, ao final de uma sequência de ELIFs não conta, também o OTHERWISE ou DEFAULT ao final de uma estrutura CASE ou SWITCH não conta.
- Estruturas de repetição no estilo FOR: 1 ponto cada.
- Estruturas de repetição no estilo REPEAT-UNTIL ou DO-WHILE: 1 ponto cada.
- Operadores lógicos binários como OR ou AND na condição de qualquer das estruturas acima: acrescenta-se 1 ponto para cada OR ou AND (ou qualquer outro operador lógico binário, se a linguagem suportar, como XOR, NAND, NOR ou IMPLIES).
- Operador lógico unário NOT: não conta.
- Chamada de sub-rotina (inclusive recursiva): não conta.
- Estruturas de seleção e repetição em sub-rotinas ou programas chamados: não contam.

Assume-se, em geral, que programas com complexidade ciclomática menor ou igual a 10 podem ser simples e fáceis de testar; com complexidade ciclomática de 11 a 20, são de médio risco em relação ao teste; de 21 a 50, são de alto risco; e que programas com complexidade acima de 50 não são testáveis. Porém, deve-se tomar cuidado com a interpretação do termo "programa". Um programa, nesse sentido, é um bloco de código único. Se ele fizer chamadas a outros blocos de código,

as estruturas desses outros blocos não serão contabilizadas na complexidade ciclomática do programa em questão. Ou seja, a complexidade é contada para uma função ou método individual e não é afetada por outras funções ou métodos chamados por ele.

A função a seguir, escrita em Python, analisa uma lista de elementos e retorna o comprimento da maior sequência de elementos repetidos em sequência (a maioria dos exemplos de código Python deste capítulo foram adaptados de Wazlawick, 2017 ou de sua lista de exercícios resolvidos):

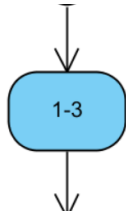
```
01 def maior_sequencia(lista):
02     elemento_anterior = None
03     contador = 1
04     maior_contador = 0
05     for elemento in lista:
06         if elemento == elemento_anterior:
07             contador += 1
08             if contador > maior_contador:
09                 maior_contador = contador
10         else:
11             contador = 1
12         elemento_anterior = elemento
13     return maior_contador
```

Por exemplo, a lista [1, 3, 3, 3, 3, 2, 2, 3, 1, 1, 3] retornaria como resultado o valor 4, pois a maior sequência de elementos contíguos repetidos nesta lista é 3, 3, 3, 3. A complexidade ciclomática será então o número de comandos de seleção ou repetição mais 1. Assim, como temos um comando FOR, um comando IF-ELSE e um comando IF, a complexidade ciclomática desta função é 4.

Na sessão seguinte é definido o grafo de fluxo de um programa, a partir do qual pode-se também calcular a complexidade ciclomática. Em caso de dúvida, na contagem de estruturas de seleção e repetição mais 1, pode-se calcular essa complexidade contando-se o número de regiões do grafo de fluxo ou, ainda, o número de nós e arestas do grafo e calculando: $2 + \text{arestas} - \text{nós}$. Nos três casos, o valor deve ser o mesmo.

13.4.2 GRAFO DE FLUXO

O *grafo de fluxo* de um programa é obtido colocando-se todos os comandos em nós e os fluxos de controle em arestas. Comandos em sequência podem ser colocados em um único nó, e estruturas de seleção e repetição devem ser representadas, via de regra, através de nós distintos com arestas que indiquem a decisão e a repetição, quando for o caso. As regras da Figura 13.3 podem ser seguidas para a criação do grafo de fluxo.

Regra	Código	Grafo
Comandos em sequência	<pre>01 <...> 02 <...> 03 <...></pre>	

IF	<pre> 01 IF condição: 02 <...> 03 <...> </pre>	<pre> graph TD Entry(()) --> 1(1) 1 -- V --> 2(2) 1 -- F --> 3(3) 2 --> Exit(()) 3 --> Exit </pre>
IF-ELSE	<pre> 01 IF condição: 02 <...> 03 ELSE 04 <...> 05 <...> </pre>	<pre> graph TD Entry(()) --> 1(1) 1 -- V --> 2(2) 1 -- F --> 34(3-4) 2 --> 5(5) 34 --> 5 5 --> Exit(()) </pre>
IF-ELIF-ELSE	<pre> 01 IF condição1: 02 <...> 03 ELIF condição2: 04 <...> 05 ELSE 06 <...> 07 <...> </pre>	<pre> graph TD Entry(()) --> 1(1) 1 -- V --> 2(2) 1 -- F --> 3(3) 3 -- V --> 4(4) 3 -- F --> 56(5-6) 2 --> 7(7) 4 --> 7 56 --> 7 7 --> Exit(()) </pre>
FOR	<pre> 01 FOR condição: 02 <...> 03 <...> </pre>	<pre> graph TD Entry(()) --> 1(1) 1 -- rep. --> 2(2) 2 --> 1 1 -- fim rep. --> 3(3) 3 --> Exit(()) </pre>

WHILE	<pre> 01 WHILE condição: 02 <...> 03 <...> </pre>	<pre> graph TD Entry(()) --> 1(1) 1 -- V --> 1 1 -- F --> 3(3) 3 --> Exit(()) </pre>
REPEAT-UNTIL ou DO-WHILE	<pre> 01 REPEAT: 02 <...> 03 UNTIL condição 04 <...> </pre>	<pre> graph TD Entry(()) --> 1(1) 1 --> 23(2-3) 23 -- F --> 1 23 -- V --> 4(4) 4 --> Exit(()) </pre>
CASE ou SWITCH	<pre> 01 CASE X: 02 a: <...> 03 b: <...> 04 <...> </pre>	<pre> graph TD Entry(()) --> 1(1) 1 -- a --> 2(2) 1 -- b --> 3(3) 2 --> 4(4) 3 --> 4 None[nenhuma das opções] --> 4 4 --> Exit(()) </pre>
CASE ou SWITCH com OTHERWISE ou DEFAULT	<pre> 01 CASE X: 02 a: <...> 03 b: <...> 04 OTHERWISE <...> 05 <...> </pre>	<pre> graph TD Entry(()) --> 1(1) 1 -- a --> 2(2) 1 -- b --> 3(3) 1 -- otherwise --> 4(4) 2 --> 5(5) 3 --> 5 4 --> 5 5 --> Exit(()) </pre>

Figura 13.3 Regras para criação de grafos de fluxo

Em relação à primeira regra, *comandos em sequência*, pode-se agrupar em um único nó quaisquer comandos, em sequência, que não sejam comandos de seleção ou repetição. Porém, quando uma sequência de comandos deste tipo terminar por um comando de decisão como IF, CASE ou SWITCH, o comando de decisão pode ser incluído como o último da sequência, já que só a partir dele será tomada a decisão. Isso não vale no caso de comandos de repetição como WHILE, FOR e REPEAT, já que quando a repetição ocorrer, alguma linha de código mais abaixo vai estar retornando o fluxo para o nó que contém o comando de repetição, mas não para a sequência de comandos que o antecedem. Assim, uma sequência de comandos terminada por um comando de repetição não pode ser agrupada com este, devendo aqueles permanecer em um nó separado do nó do comando de repetição.

Em relação ao comando ELSE, verifica-se que ele não toma nenhuma decisão, ele apenas funciona como uma espécie de rótulo que diz para onde o fluxo deve ir após a decisão do IF ser considerada falsa. Assim, os comandos subordinados ao ELSE são executados imediatamente após o fluxo chegar no ELSE. Dessa forma, o else pode ser incluído em um único nó, como o primeiro da sequência de comandos que estão subordinados a ele. Em caso de dúvida é melhor deixar os comandos em nós separados, pois isso não afeta o resultado final.

Expressões booleanas compostas com OR podem ser tratadas como instruções à parte, como na primeira linha da **Figura 13.4**. No passo 1, se *cond1* for falsa, então o fluxo vai para 1a, onde *cond2* será testada. Se no passo 1 *cond1* for verdadeira, então o fluxo vai diretamente para o passo 2. Já no nodo 1a, se *cond2* for verdadeira, o fluxo vai para o passo 2, senão vai para o passo 3-4.

Regra	Código	Grafo
OR	<pre> 01 IF cond1 OR cond2: 02 <...> 03 ELSE: 04 <...> 05 <...> </pre>	<pre> graph TD Entry(()) --> 1((1)) 1 -- V --> 2((2)) 1 -- F --> 1a((1a)) 1a -- V --> 2 1a -- F --> 34((3-4)) 2 --> 5((5)) 34 --> 5 5 --> Exit(()) </pre>

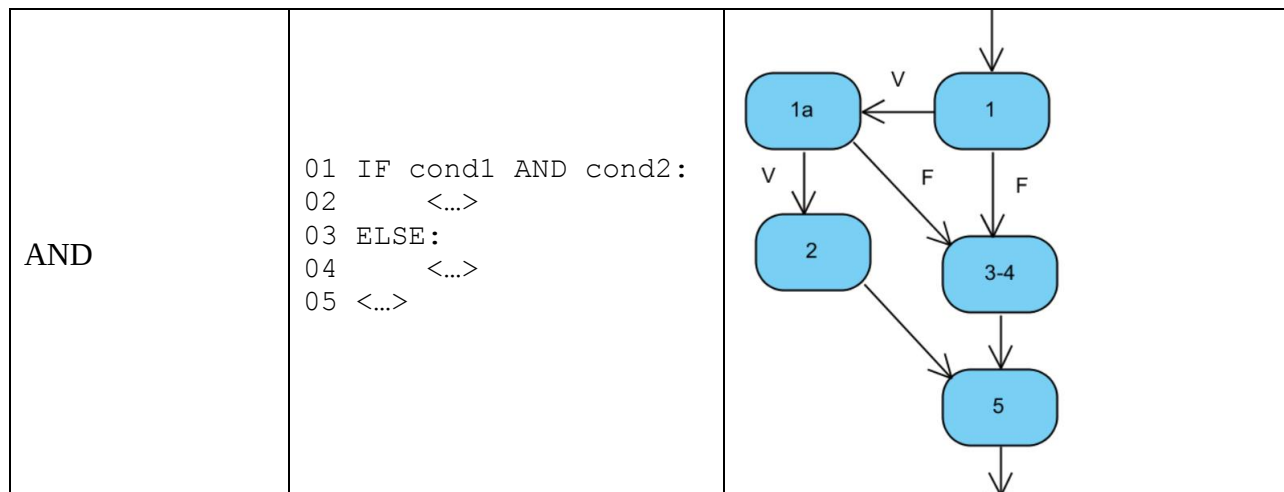


Figura 13.4 Regras para criação de grafos de fluxo com operadores booleanos binários

Simetricamente, uma condição com AND deve ser tratada como na segunda linha da Figura 13.4. No passo 1, se *cond1* for verdadeira, então o fluxo vai para 1a, onde *cond2* será testada. Se no passo 1 *cond1* for falsa, então o fluxo vai diretamente para o passo 3-4. Já no nodo 1a, se *cond2* for verdadeira, o fluxo vai para o passo 2, senão vai para o passo 3-4.

Assim, aplicando as regras ao programa mostrado anteriormente, será obtido um grafo de fluxo semelhante ao da Figura 13.5. Na figura, é possível ver que existem quatro regiões neste grafo: uma é externa ao grafo, outra fica compreendida entre os nós 5, 6, 7-8 e 12, outra entre os nós 7-8, 9 e 12, e por fim, outra compreendida entre os nós 6, 7-8, 9, 10-11 e 12. Essas quatro regiões correspondem, portanto ao número da complexidade ciclomática.

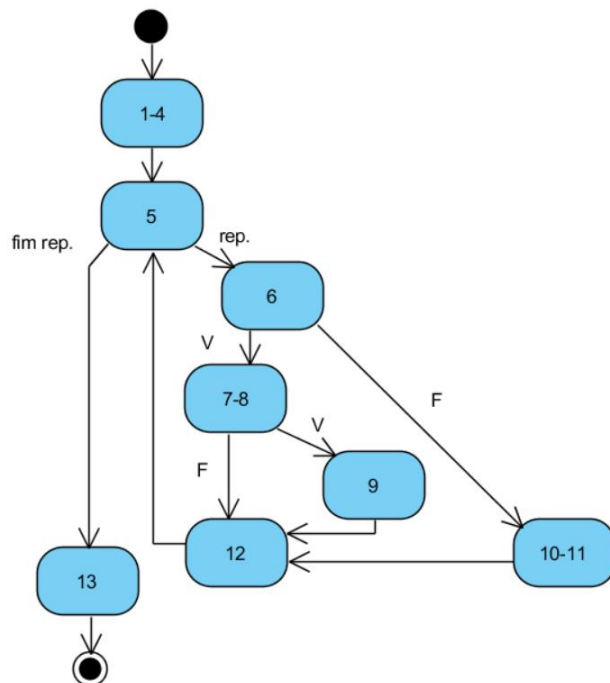


Figura 13.5 Grafo de fluxo de um programa que identifica a maior sequência de valores contíguos iguais em uma lista

Outra forma de calcular essa complexidade, caso se tenha dúvida com as duas anteriores, é contar o número de arestas do grafo, subtrair o número de nodos e somar 2. No caso da **Figura 13.5**, o grafo possui 12 arestas e 10 nodos (contando com os pseudonodos inicial e final). Portanto, sua complexidade ciclomática é $12 - 10 + 2 = 4$.

Observa-se que o uso de comandos como GOTO pode fazer com que o grafo de fluxo deixe de ser planar, e assim, ele não terá mais regiões definidas. Para construir o grafo de fluxo com estes comandos, deve-se ter em mente que eles em si não são comandos de decisão e, portanto, não adicionam complexidade ciclomática. Mas possivelmente, haverá associado a cada um deles um comando de seleção (IF) que, caso ative o GOTO levará o fluxo de programa para uma posição diferente do programa.

Certamente que o comando GOTO, já há bastante tempo é considerado inadequado para programas em linguagens de alto nível, mas técnicas de programação recentes têm consagrado o uso do comando BREAK para forçar a saída de estruturas de repetição como FOR e WHILE. Em algumas situações específicas, é considerado mais adequado usar o BREAK do que criar estruturas de repetição mais complexas sem BREAK.

Por exemplo, o programa abaixo cria uma lista com seis números aleatórios entre 1 e 50, ordenados de forma crescente, simulando, assim, o sorteio dos números da Mega-Sena. Esse programa usa dois comandos BREAK para interromper o FOR, quando as condições testadas nos IF forem verdadeiras. Por outro lado, ele usa a estrutura FOR-ELSE, típica de Python, que funciona da seguinte forma: se o FOR for interrompido por um BREAK durante a execução, então o código subordinado ao ELSE não é executado. Se, por outro lado, o FOR terminar todas as repetições sem ser interrompido por um BREAK então o código subordinado ao ELSE será executado.

```
01 while len(sorteados) < 6:
02     sorteado = randint(1, 50)
03     for posição in range(len(sorteados)):
04         número = sorteados[posição]
05         if número == sorteado:
06             break
07         if número > sorteado:
08             sorteados.insert(posição, sorteado)
09             break
10     else:
11         sorteados.append(sorteado)
12 print(sorteados)
```

A complexidade ciclomática deste programa é 5, porque existem quatro comandos de decisão, sendo um WHILE, um FOR-ELSE e dois IF. Seu grafo de fluxo é mostrado na **Figura 13.6**.

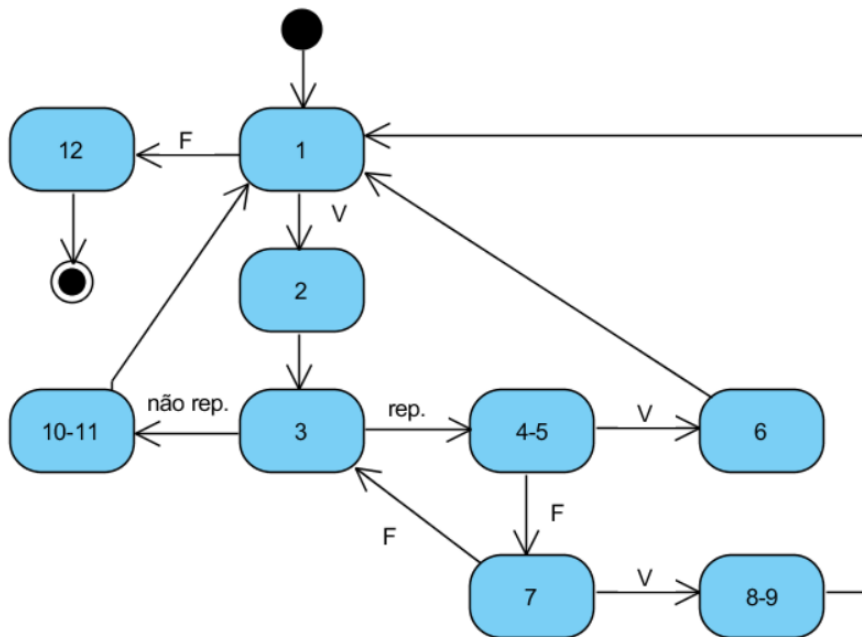


Figura 13.6 Grafo de fluxo de um programa que sorteia os números da Mega-Sena

Como se pode observar, o grafo conta com 5 regiões, correspondendo ao número da complexidade ciclômática indicado com a técnica da contagem de estruturas de seleção e repetição. Os comandos BREAK nas linhas 6 e 9, quando executados, interrompem o FOR e, como não há mais comandos subordinados ao WHILE mais externo, eles retornam à linha 1, para que a condição do WHILE seja novamente testada.

13.4.3 CAMINHOS INDEPENDENTES

O valor da complexidade ciclômática indica número *máximo* de execuções *necessárias* para exercitar todos os comandos do programa. Assim, se um programa tem complexidade ciclômática 4, por exemplo, isso significa que no máximo quatro casos de teste exercitarão todas as condições e todas as arestas. Esse valor é um máximo porque alguns caminhos de execução deste fluxo podem ser impossíveis de testar, como veremos adiante.

Observe que a execução de todos os comandos (nós) não vai necessariamente testar os valores verdadeiro e falso de todas as condições possíveis. Por exemplo, é possível passar por todos os nós do grafo da **Figura 13.6** sem nunca passar pela aresta que vai do nó 7 ao nó 3. Se todas as vezes o nó 7 levar ao nó 8-9, é possível percorrer todos os nós sem nunca passar pela aresta que vai de 7 a 3.

Assim, uma abordagem mais adequada para o teste seria exercitar não apenas todos os nós, mas todas as *arestas* do grafo de fluxo. Isso é feito pela determinação dos *caminhos independentes* do grafo, que são possíveis navegações do início ao fim do grafo.

O conjunto dos caminhos independentes pode ser definido através do seguinte algoritmo:

1. Inicialize o conjunto dos caminhos independentes com um caminho qualquer do início ao fim do grafo, preferencialmente o caminho mais curto, ou seja, o que passe pela menor quantidade de arestas possível.
2. Enquanto for possível, adicione ao conjunto dos caminhos independentes outros caminhos que

passem por pelo menos uma aresta na qual nenhum dos caminhos anteriores passou antes. Aqui também deve-se dar preferência a caminhos possíveis que minimizem a quantidade de novas arestas adicionadas de cada vez.

Retornando ao exemplo da **Figura 13.5**, o conjunto dos caminhos independentes deste grafo pode ser definido como:

- O caminho $c_1 = \langle 1-4, 5, 13 \rangle$
- O caminho $c_2 = \langle 1-4, 5, 6, 10-11, 12, 5, 13 \rangle$
- O caminho $c_3 = \langle 1-4, 5, 6, 7-8, 12, 5, 13 \rangle$
- O caminho $c_4 = \langle 1-4, 5, 6, 7-8, 9, 12, 5, 13 \rangle$

A complexidade ciclomática define o número *máximo* de testes necessários para passar por todas as arestas. Com quatro testes, como mostrado, é possível exercitar todos os comandos e todas as possíveis condições das estruturas de seleção e repetição. Apesar disso, nada impede que mais testes sejam feitos, se assim for desejado.

Já o exemplo da **Figura 13.6** conta com cinco caminhos independentes, já que sua complexidade ciclomática é 5:

- $c_1 = \langle 1, 12 \rangle$
- $c_2 = \langle 1, 2, 3, 10-11, 1, 12 \rangle$
- $c_3 = \langle 1, 2, 3, 4-5, 6, 1, 12 \rangle$
- $c_4 = \langle 1, 2, 3, 4-5, 7, 8-9, 1, 12 \rangle$
- $c_5 = \langle 1, 2, 3, 4-5, 7, 3, 10-11, 1, 12 \rangle$

Observe que até o caminho c_4 todos os comandos já haviam sido executados, mas faltava a aresta do nodo 3 para o nodo 7, que só é adicionada no caminho c_5 .

Como o algoritmo de construção deste grafo não é determinístico em relação à escolha das arestas a serem adicionadas em cada passo, resultados diferentes podem ser obtidos para o conjunto de caminhos mínimos, o que usualmente não afeta os resultados dos testes. Inclusive, como mostrado adiante, por vezes alguns caminhos criados com essa técnica podem se mostrar impossíveis de percorrer devido à lógica do programa e assim, outras escolhas de caminhos terão que ser tomadas para gerar um conjunto que possa ser percorrido.

13.4.4 CASOS DE TESTE

Resta ainda definir os *casos de teste* (Myers, Sandler, Badgett & Thomas, 2004), ou seja, quais dados de entrada levam o programa a executar os caminhos independentes e qual é a saída esperada do programa para cada um desses casos.

Considerando novamente a função *maior_sequencia* da **Seção 13.4.1**, cujo grafo é mostrado na **Figura 13.5**, a única entrada dessa função é a variável *lista*, da qual se deseja obter o comprimento da maior sequência contígua de valores iguais. Por outro lado, a saída da função é a variável *maior_contador*. De forma geral, deve acontecer o seguinte:

- Para exercitar o caminho $c_1 = \langle 1-4, 5, 13 \rangle$, o FOR da linha 5 deve desviar diretamente para a linha 13, sem efetuar nenhuma repetição de seus comandos subordinados. Para que isso aconteça, é necessário que *lista* seja vazia, ou seja, $lista == []$. Neste caso, espera-se que a variável de saída, *maior_contador*, tenha valor 0, pois não há nenhuma sequência de elementos na lista.
- Para exercitar o caminho $c_2 = \langle 1-4, 5, 6, 10-11, 12, 5, 13 \rangle$, é necessário que o FOR da linha 5 execute uma única vez; para isso *lista* deve conter um único elemento. Em seguida, a condição do IF da linha 06 deve ser falsa, para que haja o desvio para o ELSE da linha 10. Para isso,

basta que o elemento de *lista* seja diferente do valor de *elemento_anterior*. Como estamos na primeira repetição do FOR, na linha 6, o valor de *elemento_anterior* será necessariamente *None*. Assim, *lista* deve conter um único elemento diferente de *None*, digamos, [5]. Neste caso, espera-se que a variável de saída, *maior_contador* tenha valor 1, pois há apenas um elemento na lista e, não importa qual ele seja, a maior (e única sequência) dentro desta lista tem comprimento 1.

- Para exercitar o caminho $c_3 = \langle 1-4, 5, 6, 7-8, 12, 5, 13 \rangle$, seria necessário que a lista tivesse um único elemento e que, na linha 6 esse elemento fosse igual a *elemento_anterior*. Para isso acontecer na primeira repetição do FOR seria necessário que a lista contivesse o valor *None*. Mas o *None* é apenas um elemento usado para indicar *valor nenhum*, e assim, ele não é considerado um valor válido para efeito da lista. Então, para conseguir exercitar essa sequência de condições do caminho c_3 , é necessário executar a repetição do FOR duas vezes. Na primeira, haverá um valor qualquer e na segunda vez um valor igual ao primeiro, para que a condição do IF no passo 6 seja verdadeira e os passos 7 e 8 sejam executados. Este caminho exige que a condição do IF na linha 8 seja falsa, para que o fluxo vá diretamente à linha 12, sem passar pela linha 9. Isso será impossível na segunda repetição, porque neste ponto o *contador* será necessariamente maior do que o *maior_contador*, já que essa é a maior sequência (2 elementos iguais) encontrada até agora, e, nesta segunda repetição o programa necessariamente deverá passar pela linha 09. Assim, para que a sequência $\langle \dots, 5, 6, 7-8, 12, \dots \rangle$ seja executada, é necessário encontrar na lista de valores uma sequência com dois valores iguais, para que a condição da linha 6 seja verdadeira, mas essa sequência deve ter sido necessariamente precedida por uma sequência maior do que dois elementos, para que a condição da linha 8 seja falsa. Assim, por exemplo, uma lista que faria executar essa sequência de comandos (dentro de uma sequência maior), poderia ser [5, 5, 5, 7, 7]. Neste caso, o valor da variável de saída, *maior_contador*, deverá ser 3, visto que a maior sequência é a que tem três números 5.
- Para exercitar o caminho $c_4 = \langle 1-4, 5, 6, 7-8, 9, 12, 5, 13 \rangle$, é necessário executar pelo menos uma repetição do FOR e então a lista deve ter pelo menos um elemento. Como a condição da linha 6 deve ser verdadeira, então deve haver pelo menos uma sequência de dois elementos iguais contíguos na lista e esta sequência deve ser maior do que a maior sequência já encontrada, ou seja, *contador* deve ser maior que *maior_contador*, para que a condição da linha 8 seja verdadeira. Assim, basta que uma lista com dois valores iguais seja usada como entrada para que a sequência desejada seja executada. Como no caso anterior, o caminho não será executado exatamente da forma como foi definido, pois haverá obrigatoriamente duas repetições do FOR, e não apenas uma. Mas o importante aqui é que as condições das linhas que contêm comandos de decisão sejam executadas com o valor verdadeiro ou falso, conforme o caminho original requer. Assim, por exemplo, este caminho poderia ser testado com a lista [5, 5]. Neste caso, o valor da variável de saída, *maior_contador*, será 2.

Um conjunto de casos de teste, então, poderia ser definido como na **Tabela 13.1**.

Tabela 13.1 Casos de teste para a função *maior_sequencia*

#	caminho	entrada	saída esperada
		<i>lista</i>	<i>maior_contador</i>
c_1	$\langle 1-4, 5, 13 \rangle$	[]	0
c_2	$\langle 1-4, 5, 6, 10-11, 12, 5, 13 \rangle$	[5]	1
c_3	$\langle 1-4, 5, 6, 7-8, 12, 5, 13 \rangle$	[5, 5, 5, 7, 7]	3

c_4	<1-4, 5, 6, 7-8, 9, 12, 5, 13>	[5, 5]	2
-------	--------------------------------	--------	---

Se executarmos os quatro testes com o programa em questão, com os comandos abaixo veremos que ele *falha* no teste do caminho c_2 .

```
assert maior_sequencia([]) == 0
assert maior_sequencia([5]) == 1
assert maior_sequencia([5, 5, 5, 7, 7]) == 3
assert maior_sequencia([5, 5]) == 2
```

Surpresa? Ocorre que, quando há um único elemento na lista, a variável *maior_contador*, que foi inicializada com 0, não é atualizada para valer 1. Ela só será atualizada se as condições da linha 9 e da linha 11 forem verdadeiras, o que não ocorre no caso da lista de um único elemento, pois neste caso, a condição da linha 9 será necessariamente falsa e não há outro lugar no programa onde *maior_contador* seja atualizado.

O defeito deste programa está no fato de que a atualização de *maior_contador* deve ocorrer tanto quando o primeiro elemento da lista é percorrido, quanto quando uma sequência maior do que as anteriores for encontrada. Para que isso ocorra, essa instrução não pode estar subordinada ao IF da linha 9, devendo passar para depois do ELSE relacionado ao IF da linha 9. O programa abaixo é, assim, uma *correção* do programa anterior, que passa em todos os testes.

```
01 def maior_sequencia(lista):
02     elemento_anterior = None
03     contador = 1
04     maior_contador = 0
05     for elemento in lista:
06         if elemento == elemento_anterior:
07             contador += 1
08         else:
09             contador = 1
10             if contador > maior_contador: # antiga linha 8 movida para cá
11                 maior_contador = contador # antiga linha 9 movida para cá
12             elemento_anterior = elemento
13     return maior_contador
```

A complexidade ciclomática do novo programa permanece inalterada (4). Mas a má notícia é que como a estrutura do programa foi alterada, será necessário refazer o seu grafo de fluxo, identificar os novos caminhos independentes e os casos de teste. Essas atividades ficam como sugestão de exercício para o leitor.

13.4.5 MÚLTIPLAS CONDIÇÕES E CAMINHOS IMPOSSÍVEIS

O critério de cobertura de todas as arestas, conforme visto anteriormente, precisa cobrir também os desvios provocados pelo uso de operadores lógicos binários em estruturas de seleção e repetição, como o programa abaixo.

```
01 # Programa que verifica se um ano é bissexto
02 print('Digite um ano e direi se é bissexto')
03 ano = int(input('Ano:'))
04 if ano % 4 == 0 and not ano % 100 == 0 or ano % 400 == 0:
05     print('é bissexto')
```

```

06 else:
07     print('não é bissexto')

```

O programa em questão verifica que o ano é bissexto se ele for divisível por 4, não divisível por 100 ou divisível por 400. Assim, a linha 4 conta com uma condição composta com três condições componentes: $ano \% 4 == 0$, $not\ ano \% 100 == 0$ e $ano \% 400 == 0$. Sabendo-se que a precedência do operador lógico AND é mais alta do que a do OR, podemos ver que as três cláusulas serão avaliadas na sequência da esquerda para a direita. Se a primeira for verdadeira, a segunda será avaliada. Se ela também for verdadeira, não será necessário avaliar a terceira. Se a primeira for falsa, não será necessário avaliar a segunda, já que a combinação das duas será necessariamente falsa. Mas será necessário então avaliar a terceira, já que se ela for verdadeira a combinação das três também será verdadeira.

O grafo de fluxo do programa é mostrado na **Figura 13.7**. O nó 4a corresponde à segunda condição da linha 4 e o nó 4b à terceira condição da linha 4.

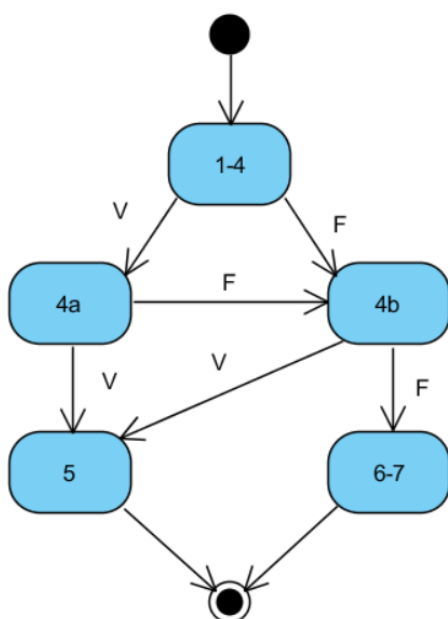


Figura 13.7 Grafo de fluxo de um programa que verifica se um ano é bissexto

Observe que como o programa pode terminar a execução tanto na linha 5 quanto na linha 7, é importante que o grafo contenha o pseudonodo de finalização e que todos os pontos possíveis de parada do programa converjam para este nodo, mesmo que ele não represente nenhuma linha de código. Isso fará com que o número de regiões do grafo coincida com a complexidade ciclomática do programa, que é 4. Observe que a contagem de estruturas de decisão, IF, OR, AND mais 1, também indica o valor 4. E a contagem das arestas, menos os nós, mais 2, dará $9-7+2$, o que também é 4.

Nesse caso, portanto, quatro testes será o máximo necessário para verificar todas as condições do programa. O conjunto de caminhos independentes desse grafo poderia ser, então, definido assim (primeira tentativa):

- c_1 : <1-4, 4a, 5>
- c_2 : <1-4, 4b, 5>

- c_3 : <1-4, 4b, 6-7>
- c_4 : <1-4, 4a, 4b, 6-7>

Poder-se-ia ainda pensar em executar o caminho <1-4, 4a, 4b, 5>, que também é possível. Mas todas as condições testadas por ele já foram testadas nos outros quatro caminhos. Como já foi dito, não há problema algum em executar mais do que os mínimos caminhos independentes. O que não se pode fazer é deixar de testar alguma condição ou linha do programa pela falta de um caminho que deveria ser testado.

Para executar o caminho c_1 é necessário que o ano seja divisível por 4 e não divisível por 100. Assim, seria possível usar valores como 4, 80, 2004 etc.

Para executar o caminho c_2 é necessário que o ano não seja divisível por 4 mas seja divisível por 400. Ops! Aqui temos um problema: como 400 é múltiplo de 4, qualquer número não divisível por 4 também não será divisível por 400. Assim, o caminho c_2 , <1-4, 4b, 5>, é considerado um *caminho impossível* de ser executado. Se a sequência <1-4, 4b> for executada é porque o número não é divisível por 4. Assim, quando se está em 4b, vindo diretamente de 1-4, a única saída possível é para 6-7, já que o número não será necessariamente divisível por 400 também. Assim, podemos reconstruir nosso conjunto de caminhos independentes, da seguinte forma (definitiva):

- c_1 : <1-4, 4a, 5>
- c_2 : <1-4, 4b, 6-7>
- c_3 : <1-4, 4a, 4b, 5>
- c_4 : <1-4, 4a, 4b, 6-7>

Já sabemos que para executar c_1 , <1-4, 4a, 5>, o ano deve ser divisível por 4 mas não por 100. Assim, o ano seria bissexto. Para executar c_2 , <1-4, 4b, 6-7>, o número não pode ser divisível por 4 nem por 400. Assim, dados de entrada para testar esse caminho seriam anos como 3, 81, 1999 etc. O resultado é um ano não bissexto.

Para executar c_3 , <1-4, 4a, 4b, 5>, o número deve ser divisível por 4, divisível por 100 e divisível por 400. Qualquer número divisível por 400 também o será por 100 e por 4. Assim, dados de entrada para testar esse caminho seriam anos como 400, 1600, 2000 etc. O resultado é um ano bissexto.

Para executar c_4 , <1-4, 4a, 4b, 6-7>, o número deve ser divisível por 4 e divisível por 100, mas não divisível por 400. Qualquer número divisível por 100 também o será por 4. Assim, dados de entrada para testar esse caminho seriam anos como 300, 1100, 1800 etc. O resultado é um ano não bissexto.

Assim, o conjunto de casos de teste para esse programa poderia ser definido conforme a **Tabela 13.2**.

Tabela 13.2 Casos de teste para o programa que identifica se um ano é bissexto

#	caminho	entrada	saída esperada
		ano	bissexto?
c_1	<1-4, 4a, 5>	4, 80, 2004	sim
c_2	<1-4, 4b, 6-7>	3, 81, 1999	não
c_3	<1-4, 4a, 4b, 5>	400, 1600, 2000	sim
c_4	<1-4, 4a, 4b, 6-7>	300, 1100, 1800	não

Pode-se observar que a execução do programa com as entradas definidas na tabela acima vai exercitar todas as combinações possíveis dos valores-verdade das condições pelo menos uma vez.

Outra situação em que surgem caminhos impossíveis é quando se tem um FOR com número fixo de repetições. Usualmente, um FOR no qual um dos limites (inferior ou superior) é dado por uma variável ou expressão contendo variável, gera dois caminhos possíveis: aquele em que a repetição ocorre pelo menos uma vez e aquele em que a repetição não ocorre nenhuma vez. Por exemplo:

```
fatorial = 1
for i in range(1, n+1):
    fatorial *= i
    print('{0}!={1}'.format(i, fatorial))
```

Neste programa, a repetição pode não ocorrer nenhuma vez se o valor de n for menor ou igual a zero. Porém, para qualquer valor de n maior do que zero, a repetição vai ocorrer pelo menos uma vez.

Mas na seguinte situação, existe apenas um caminho possível:

```
print('Pagamento pelo jogo de xadrez')
grãos = 0
for i in range(64):
    grãos += 2**i
print('Grãos de arroz:', grãos)
```

O caminho no qual a repetição não é feita nenhuma vez é um caminho impossível, já que o FOR foi definido para executar 64 repetições em qualquer situação.

Esse tipo de situação também pode ocorrer com estruturas WHILE com condição *True*. Por exemplo, o programa abaixo calcula a evolução de uma população em extinção, considerando nascimentos e mortes anualmente, e verificando em que ano essa população chegaria ao valor mínimo definido:

```
01. TAXA_MORTES = 0.06
02. TAXA_NASCIMENTOS = 0.01
03. POPULAÇÃO_MÍNIMA = 100000
04. print('Evolução da população de dodôs.')
05. ano = 1600
06. população = 1000000
07. while True:
08.     mortos = round(população*TAXA_MORTES)
09.     nascidos = round(população*TAXA_NASCIMENTOS)
10.     print('Ano: {0}. População: {1}. Nascidos: {2}. Mortos: {3}.'
11.         .format(ano, população, nascidos, mortos))
12.     if população < POPULAÇÃO_MÍNIMA:
13.         break
14.     população += nascidos-mortos
15.     ano += 1
```

O grafo de fluxo deste programa é mostrado na **Figura 13.8**.

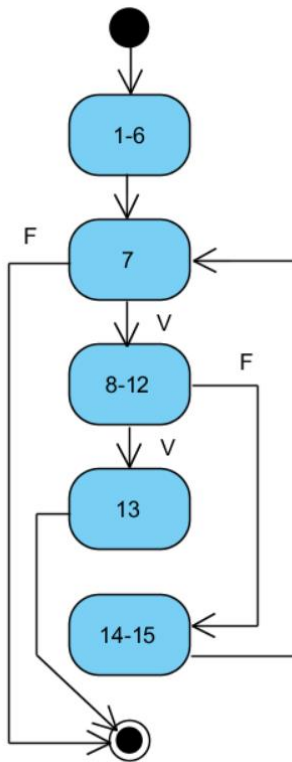


Figura 13.8 Grafo de fluxo de um programa que calcula a evolução de uma população

Em função da forma como a linha 7 foi escrita, embora ela contenha um comando WHILE, a condição nunca será falsa e assim, a aresta que vai do nó 7 ao pseudonó final nunca poderá ser percorrida. Na prática, o ponto de decisão sobre continuar ou não a repetição foi transferido do WHILE na linha 7 para o IF da linha 12, já que se este for verdadeiro, o programa segue para a linha 13 com um BREAK, que é o comando que efetivamente encerra a repetição.

A complexidade ciclomática deste programa seria 3, mas como a aresta do nó 7 ao nó final é impossível de ser trilhada, na prática, sua complexidade pode ser considerada como 2.

Também os comandos IF e CASE podem apresentar caminhos impossíveis nem sempre fáceis de serem identificados. Um exemplo de IF com caminho impossível fácil de verificar é:

```

01 if 1 == 2:
02     print('essa linha nunca será executada')
  
```

Qualquer programador com um mínimo de experiência poderia perceber que a linha 2 nunca será executada neste programa, pois o número 1 nunca será igual a 2.

Mas há situações em que é muito mais difícil (quase impossível) saber se um comando será ou não executado, mesmo no caso de um programa muito simples como a função a seguir:

```

01. def fermat(x, a, b, c):
02.     if a**x + b**x == c**x:
03.         print ('O último teorema de Fermat é verdadeiro para', x, a, b, c)
  
```

Essa função testa o *último teorema de Fermat*, uma conjectura de que não existe nenhum conjunto de inteiros positivos a , b , c e x , com $x \geq 3$, tal que $a^x + b^x = c^x$. Para saber se a linha 3 poderá ser

executada, é necessário saber quais os valores de x , a , b e c que tornam a equação verdadeira.

Esse teorema levou 359 anos para ser provado matematicamente (Singh, 1998), e a conclusão é que tais valores não existem. Assim, caso o programa acima tivesse sido escrito na época de Fermat, teríamos levado mais de três séculos e meio só para saber se o caminho que passa pela linha 3 é impossível.

13.4.6 LIMITAÇÕES

O teste estrutural costuma ser aplicado para verificar defeitos no código, mas esse tipo de teste não é capaz de identificar, por exemplo, se o programa foi bem especificado. Ou seja, o teste vai verificar se o programa se comporta de acordo com a especificação dada, mas não necessariamente se ele se comporta de acordo com a especificação *esperada*. Então, essa técnica não pode ser convenientemente usada para validar sistemas, apenas para verificar defeitos.

Outra limitação conhecida do teste estrutural é o fato de que funcionalidades ausentes não são testadas, pois a técnica preconiza apenas o teste daquilo que existe no programa. Ou seja, se algum comando *deveria* ter sido incluído no código, mas não foi, o teste estrutural não será capaz de identificar isso. Considere, por exemplo, a função abaixo que verifica se um número é primo analisando se algum número entre 2 e metade do número é seu divisor inteiro. Se tal divisor for encontrado, então o número não é primo. Porém, se neste intervalo não for encontrado nenhum divisor, então a função executa a linha 5, e retorna o valor *verdadeiro*, indicando que o número, sim, é primo:

```
01 def é_primo(número):  
02     for divisor in range(2, número//2+1):  
03         if número % divisor == 0:  
04             return False  
05     return True
```

Primeiramente, o grafo de fluxo é construído e apresentado na Figura 13.9.

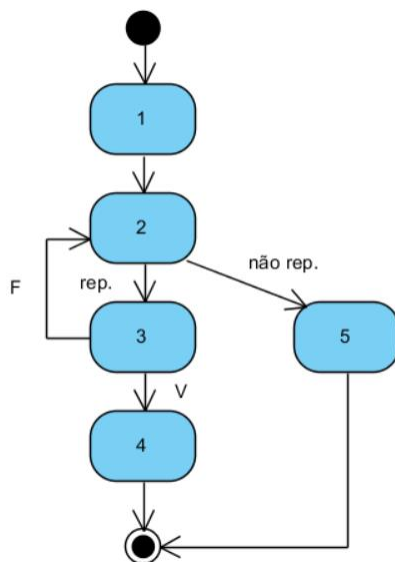


Figura 13.9 Grafo de fluxo de uma função que verifica se um número dado é primo

Observando este grafo, pode-se concluir que a complexidade ciclomática é 3. Assim, três caminhos independentes serão construídos:

- c_1 : <1, 2, 5>. A função *range* de Python com dois parâmetros n e m , produz uma lista de números (*range* ou intervalo) que vai de n a $m-1$. Assim, para executar essa sequência de passos é necessário que o m seja pelo menos uma unidade maior do que n . Assim, para obter a situação de nenhuma repetição do FOR, *número* deve valer 2. Pois neste caso a expressão será *range*(2, 2) e o FOR não vai executar nenhuma vez. O mesmo resultado pode ser conseguido com *número* = 3, já que a divisão aplicada no segundo parâmetro do *range* é a divisão inteira, sem resto. Como resultado, nos dois casos, o *número* é primo.
- c_2 : <1, 2, 3, 4>. Para percorrer este caminho, o FOR deve ser executado uma única vez. Nesta única execução do FOR a condição do IF da linha 3 deve ser verdadeira, ou seja, 2 deve ser divisor inteiro de *número*. Assim, apenas o valor 4 para a variável *número* fará com que esse caminho seja executado. Isso faz o programa terminar na linha 4 e como resultado, o *número* não é primo.
- c_3 : <1, 2, 3, 2, 4> ou <1, 2, 3, 2, 5>. A única aresta que falta adicionar aos caminhos independentes é a que retorna do nó 3 para o nó 2. Para isso acontecer, é necessário que pelo menos duas repetições do FOR aconteçam. Isso será obtido para *números* a partir de 4. Será indiferente testar, a partir daí, se o fluxo termina na linha 4 ou na linha 5 porque as duas situações já foram testadas nos caminhos anteriores. Se quisermos executar o caminho <1, 2, 3, 2, 4>, devemos usar *número* = 6, que faz a repetição ocorrer duas vezes e sair pela linha 4 indicando *número* não primo. Por outro lado, se quisermos testar <1, 2, 3, 2, 5>, podemos usar *número* = 5, pois isso faz a repetição ocorrer duas vezes e termina a execução na linha 5 indicando que o *número* é primo.

Assim, os casos de teste para este programa podem ser definidos como na **Tabela 13.3**.

Tabela 13.3 Casos de teste para um programa que verifica se um número é primo

#	caminho	entrada	saída esperada
		<i>número</i>	<i>primo?</i>
c_1	<1, 2, 5>	2	sim
c_2	<1, 2, 3, 4>	4	não
c_3	<1, 2, 3, 2, 4>	6	não
c_3 (alt.)	<1, 2, 3, 2, 5>	5	sim

Assim, o seguinte programa poderia testar a função de acordo com esses casos de teste:

```
assert é_primo(2)
assert not é_primo(4)
assert not é_primo(6)
assert é_primo(5)
```

Pode-se agora executar os testes e verificar que a função passa em todos eles. É possível concluir então que a função está livre de defeitos? Infelizmente não, porque, como comentado anteriormente, a técnica de teste estrutural não verifica a ausência de código que deveria existir.

O fato é que a função não verifica corretamente números menores do que 2, inclusive negativos. Para qualquer número menor do que 2 o FOR não será executado nenhuma vez e, assim, o

programa vai considerar que o número é primo. Não faz sentido afirmar que 0, 1, -1, -2, -3, -4 etc. sejam primos. Assim, o programa ainda tem um defeito importante que deveria ser corrigido adicionando-se no início dele, possivelmente, uma verificação de forma que, caso o número seja menor do que 2, a função deve retornar *falso*. Este tipo de problema é bastante reduzido com o uso da técnica funcional, apresentada na seção seguinte.

Uma última limitação do teste estrutural relaciona-se ao fato de que o teste estrutural só pode ser produzido *depois* que o código está escrito, o que não habilita seu uso com a técnica de desenvolvimento dirigido pelo teste (TDD), preconizada pelos métodos ágeis, que sugere que os casos de teste sejam definidos antes de se escrever o código.

Apesar dessas limitações, a técnica é relevante e seu uso é importante para a detecção de vários tipos de defeitos no software, especialmente nas suas unidades mais básicas, pois ele é usado para garantir que todos os comandos e condições lógicas sejam executados pelo menos uma vez. Em especial, quando se tem código não confiável, com possibilidade de defeitos propositais como vírus ou sabotagens, o teste estrutural, que verifica todas as linhas e condições do programa, será capaz de detectar essas anomalias, para as quais o teste funcional poderia falhar.

Como exemplo, imagine que se tenha uma função que calcula o fatorial de um número. A técnica de teste funcional não observa o código, mas apenas os parâmetros e resultados. Assim, pela técnica funcional, a função seria testada com valores como 0, 1, 2 e talvez 5, 8, 12 etc. Ou seja, alguns valores limítrofes como 0 e 1 e valores quaisquer aleatoriamente selecionados no domínio no qual a função é definida. Isso seria decidido pelo analista de teste sem que o código fosse sequer observado.

Agora, imagine que o código efetivamente testado seja o seguinte:

```
def fatorial(n):
    if n == 31:
        return 0
    if n == 0:
        return 1
    else:
        return n * fatorial(n-1)

assert fatorial(0) == 1
assert fatorial(1) == 1
assert fatorial(2) == 2
assert fatorial(5) == 120
assert fatorial(8) == 40320
assert fatorial(12) == 479001600
```

O programa vai passar em todos os testes realizados. Mas, observe que logo no início do código há uma linha que verifica se n é igual a 31 e, caso seja, retorna 0, que não é o resultado esperado. Essa linha foi introduzida propositalmente no código para que ele falhe para um valor arbitrário que possivelmente não seria testado no teste funcional, pois quem poderia imaginar que uma função que calcula corretamente todos os fatoriais falha única e exclusivamente para o valor 31?

Porém, com o uso da técnica estrutural, essa linha vai ter que necessariamente testada, pois ela produz alguma coisa. Assim, o teste estrutural vai obrigar a execução da função com o argumento 31 e será detectado que o valor resultante não é o esperado.

13.5 Teste Funcional

O teste estrutural é adequado quando se pretende verificar a estrutura de um programa. Mas, em várias situações, a necessidade consiste em verificar a funcionalidade de um programa independentemente de sua estrutura interna. Um programa pode ter uma especificação ou comportamento esperado, em geral elaborado em um contrato de operação de sistema, na forma de pré e pós condições, e o que se deseja saber é se ele efetivamente cumpre esse contrato ou especificação.

O teste funcional considera apenas os valores de entrada de um programa e suas saídas, conforme uma especificação. Como não se pode, usualmente, testar todos os valores possíveis de entrada para um programa, uma amostragem adequada deve ser feita. Basicamente duas técnicas determinam quais devem ser estes valores: *classes de equivalência* e *análise de valor limítrofe*. Essas técnicas são explicadas nas subseções 13.5.1 e 13.5.2 com exemplos em nível de teste de unidade. Mais adiante, na Subseção 13.5.3 são apresentados exemplos de aplicação do teste funcional em nível de sistema, usando casos de uso.

13.5.1 PARTICIONAMENTO DE EQUIVALÊNCIA

Um dos princípios do teste funcional é a identificação de situações equivalentes. Por exemplo, se um programa aceita um conjunto de dados (normalidade) e rejeita outro conjunto (exceção), pode-se dizer que existem duas *classes de equivalência* para os dados de entrada do programa: a dos dados aceitos e a dos dados rejeitados. Pode ser impossível testar todos os elementos de cada classe de equivalência, até porque essas classes podem ter infinitos elementos. Então, o *particionamento de equivalência* vai determinar que *pelo menos* um elemento de cada classe de equivalência seja testado.

Classicamente, a técnica de particionamento de equivalência considera a divisão das entradas possíveis da seguinte forma (Myers, Sandler, Badgett & Thomas, 2004):

- Se as entradas válidas são especificadas como um *intervalo de valores* (por exemplo, de 10 a 20), então são definidas uma classe válida, de 10 a 20, e duas classes inválidas: menor do que 10 e maior do que 20.
- Se as entradas válidas são especificadas como uma *quantidade de valores* (por exemplo, uma lista com 5 elementos), então são definidas uma classe válida (lista com 5 elementos) e duas inválidas (lista com menos de 5 elementos e lista com mais de 5 elementos).
- Se as entradas válidas são especificadas como um *conjunto de valores aceitáveis* que podem ser tratados de forma diferente (por exemplo, as *strings* “masculino” e “feminino”), então é definida uma classe válida para cada uma das formas de tratamento e uma classe inválida para outros valores quaisquer.
- Se as entradas válidas são especificadas como uma condição do tipo “deve ser de tal forma” (por exemplo, uma restrição sobre os dados de entrada como “o ISBN do livro não deve constar na base de dados”), então devem ser definidas uma classe válida (livros cujo ISBN não consta na base de dados) e uma inválida (livros cujo ISBN já consta na base de dados).

As classes válidas devem ser definidas não só em termos de restrições sobre as entradas, mas também em função dos resultados a serem produzidos. Se a operação a ser testada puder ter dois comportamentos possíveis em função do valor de um dos parâmetros, vão existir duas classes distintas de valores válidos para aquele parâmetro, um para cada comportamento possível.

Por exemplo, considere o modelo conceitual da Figura 13.10 e uma operação de sistema *adicionarItem(idCompra, isbn, quantidade)*. O comportamento desta operação é especificado da seguinte forma:

- Precondições:

- O parâmetro *idCompra* se refere a uma compra existente e que não esteja fechada.
- O parâmetro *isbn* se refere a um livro que está cadastrado.
- O parâmetro *quantidade* é um inteiro positivo (maior do que zero).
- O parâmetro *quantidade* deve ser menor ou igual ao atributo *estoque* do livro.
- Pós condições:
 - Se o livro já consta na compra, a quantidade do item referente é incrementada com o valor de *quantidade* e o valor de *estoque* do livro é decrementado no mesmo valor.
 - Se o livro não consta na compra, é criado um item correspondente e sua quantidade definida como *quantidade* e o valor de *estoque* do livro é decrementado no mesmo valor.

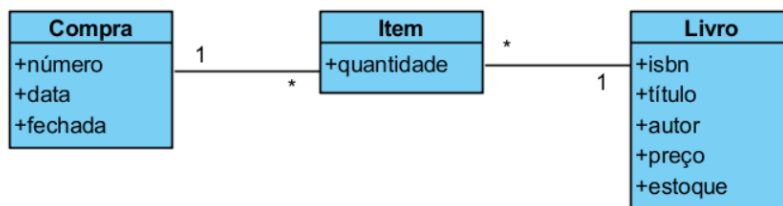


Figura 13.10 Modelo conceitual de referência

Assim, de acordo com a técnica de partição de equivalência, deve-se examinar as classes válidas e inválidas para cada um dos três parâmetros de entrada considerando especialmente as precondições e pós condições. Observa-se que necessariamente um parâmetro deve ter pelo menos uma classe válida, mas não necessariamente classes inválidas. Além disso, para cada regra (precondição) associada a este parâmetro, pelo menos uma classe inválida deve ser definida. Por exemplo, no caso do *isbn* há uma classe válida para livros cadastrados e uma classe inválida para valores de *isbn* que não correspondem a livros cadastrados.

Quando a precondição é composta, como no caso da regra para o parâmetro *idCompra*, deve-se criar uma classe inválida para cada situação referenciada. No caso de *idCompra*, a precondição é composta por duas partes interligadas por um "e". Observe: "o *idCompra* se refere a uma compra existente" e "que não esteja fechada". Assim, além da classe válida obrigatória para este parâmetro, haverá duas classes inválidas: *idCompra* que não corresponde a nenhuma compra cadastrada e *idCompra* que corresponde a uma compra cadastrada, mas que já está fechada.

Em relação às pós condições, observa-se que há dois comportamentos possíveis para a função, que dependem do fato do livro em questão estar ou não na compra. São, portanto, duas classes válidas para o parâmetro *isbn*: a dos livros que já constam na compra e a dos livros que não constam na compra. Em resumo, então, as classes válidas e inválidas para cada um dos atributos são apresentadas na **Tabela 13.4**. Para facilitar as referências posteriores, cada classe é identificada por um prefixo "v" para válida e "i" para inválida, seguido do número de ordem do parâmetro e do número de ordem da classe.

Tabela 13.4 Classes válidas e inválidas para os parâmetros da função *inserirItem*

Parâmetro	Classes válidas	Classes Inválidas
<i>idCompra</i>	v1.1: <i>idCompra</i> é igual ao atributo <i>número</i> de uma compra existente cujo atributo <i>fecha</i> é falso	i1.1: <i>idCompra</i> não corresponde a nenhuma compra existente i1.2: <i>idCompra</i> é igual ao atributo <i>número</i> de uma compra existente cujo atributo <i>fecha</i> é verdadeiro
<i>isbn</i>	v2.1: <i>isbn</i> é igual ao atributo <i>isbn</i> de um livro que consta na compra v2.2: <i>isbn</i> é igual ao atributo <i>isbn</i> de um livro que não consta na compra	i2.1: <i>isbn</i> não corresponde a nenhum livro cadastrado
<i>quantidade</i>	v3.1: <i>quantidade</i> é maior do que zero e menor do que o atributo <i>estoque</i> do livro	i3.1: <i>quantidade</i> é menor ou igual a zero i3.2: <i>quantidade</i> é maior do que o atributo <i>estoque</i> do livro

Os casos de teste são definidos considerando-se todas as classes de equivalência. Cada caso vai atribuir a cada um dos três parâmetros um valor pertencente a uma classe válida ou inválida. Usualmente, todas as combinações possíveis de classes válidas devem ser testadas, consistindo em testes de *sucesso*. No exemplo corrente, o primeiro parâmetro tem uma classe válida, o segundo tem duas e o terceiro tem uma. Assim, dois casos de teste de sucesso devem ser definidos:

- Primeiro caso de teste de sucesso com: v1.1, v2.1 e v3.1. Como resultado, o valor da quantidade do item é incrementado e o estoque do livro decrementado no mesmo valor.
- Segundo caso de teste de sucesso com: v1.1, v2.2 e v3.1. Como resultado, deve ser criado um novo item, com *quantidade* igual ao parâmetro de mesmo nome e estoque do livro decrementado no mesmo valor.

Se todos os parâmetros tivessem apenas uma classe válida, então haveria um único caso de teste de sucesso. Mas se, por exemplo, o primeiro parâmetro tivesse três classes válidas, o segundo duas e o terceiro quatro, haveria $3 \times 2 \times 4 = 24$ casos de teste de sucesso.

Já os testes de falha, são feitos, usualmente com uma classe inválida de um dos parâmetros combinada com classes válidas dos demais parâmetros. Combinações de classes inválidas de parâmetros diferentes em um mesmo método, não são necessariamente proibidas, mas dificilmente elas são úteis, pois, havendo dois parâmetros inválidos em uma chamada da função o programa provavelmente vai parar quando encontrar o primeiro erro e nem chegará a examinar o segundo. Além disso, algumas vezes, pode ser impossível combinar valores inválidos de diferentes atributos. No exemplo, a classe i2.1 é incompatível com i3.1 e i3.2, pois se o *isbn* não corresponde a nenhum livro cadastrado, não é possível verificar a quantidade em estoque correspondente.

Assim, os testes de falha para este exemplo podem ser definidos como:

- Primeiro caso de teste de falha com: i1.1, v2.2 (se a compra não existe, o livro não pode estar nela) e v3.1. Como resultado, é apresentado o erro de que não existe compra com o *idCompra* informado.
- Segundo caso de teste de falha com: i1.2, v2.1 e v3.1. Aqui, tenta-se incrementar um livro existente em uma compra fechada. O resultado é um erro de compra fechada.
- Terceiro caso de teste de falha com: i1.2, v2.2 e v3.1. Aqui, tenta-se adicionar um livro a uma compra fechada. O resultado é um erro de compra fechada.
- Quarto caso de teste de falha com: v1.1, i2.1 e v3.1. Aqui tenta-se informar um livro não

cadastrado. O resultado é um erro de livro não cadastrado.

- Quinto caso de teste de falha com: v1.1, v2.1, i3.1. Aqui, tenta-se incrementar um livro na compra com quantidade não positiva.
- Sexto caso de teste de falha com: v1.1, v2.2, i3.1. Aqui, tenta-se adicionar um livro na compra com quantidade não positiva.
- Sétimo caso de teste de falha com: v1.1, v2.1, i3.2. Aqui, tenta-se incrementar um livro na compra com quantidade acima do estoque.
- Oitavo caso de teste de falha com: v1.1, v2.2, i3.2. Aqui, tenta-se adicionar um livro na compra com quantidade acima do estoque.

A **Tabela 13.5** apresenta o resumo dos casos de teste conforme definidos acima.

Tabela 13.5 Casos de teste para a função *inserirItem*

Tipo	#	<i>idCompra</i>	<i>isbn</i>	<i>quantidade</i>	Resultado
Sucesso	s1	v1.1: <i>idCompra</i> é igual ao atributo <i>número</i> de uma compra existente cujo atributo <i>fechada</i> é falso	v2.1: <i>isbn</i> é igual ao atributo <i>isbn</i> de um livro que consta na compra	v3.1: <i>quantidade</i> é maior do que zero e menor do que o atributo <i>estoque</i> do livro	O valor da <i>quantidade</i> do item é incrementado e o <i>estoque</i> do livro decrementado no mesmo valor
	s2	v1.1: <i>idCompra</i> é igual ao atributo <i>número</i> de uma compra existente cujo atributo <i>fechada</i> é falso	v2.2: <i>isbn</i> é igual ao atributo <i>isbn</i> de um livro que não consta na compra	v3.1: <i>quantidade</i> é maior do que zero e menor do que o atributo <i>estoque</i> do livro	Foi criado um novo item, com <i>quantidade</i> igual ao parâmetro de mesmo nome e <i>estoque</i> do livro decrementado no mesmo valor
Falha	f1	i1.1: <i>idCompra</i> não corresponde a nenhuma compra existente	v2.2: <i>isbn</i> é igual ao atributo <i>isbn</i> de um livro que não consta na compra	v3.1: <i>quantidade</i> é maior do que zero e menor do que o atributo <i>estoque</i> do livro	Erro de compra inexistente
	f2	i1.2: <i>idCompra</i> é igual ao atributo <i>número</i> de uma compra existente cujo atributo <i>fechada</i> é verdadeiro	v2.1: <i>isbn</i> é igual ao atributo <i>isbn</i> de um livro que consta na compra	v3.1: <i>quantidade</i> é maior do que zero e menor do que o atributo <i>estoque</i> do livro	Erro de compra fechada
	f3	i1.2: <i>idCompra</i> é igual ao atributo <i>número</i> de uma compra existente cujo atributo <i>fechada</i> é verdadeiro	v2.2: <i>isbn</i> é igual ao atributo <i>isbn</i> de um livro que não consta na compra	v3.1: <i>quantidade</i> é maior do que zero e menor do que o atributo <i>estoque</i> do livro	Erro de compra fechada
	f4	v1.1: <i>idCompra</i> é igual ao atributo <i>número</i> de uma compra existente cujo atributo <i>fechada</i> é falso	i2.1: <i>isbn</i> não corresponde a nenhum livro cadastrado	v3.1: <i>quantidade</i> é maior do que zero e menor do que o atributo <i>estoque</i> do livro	Erro de livro não cadastrado
	f5	v1.1: <i>idCompra</i> é igual ao atributo <i>número</i> de uma compra existente cujo atributo <i>fechada</i> é falso	v2.1: <i>isbn</i> é igual ao atributo <i>isbn</i> de um livro que consta na compra	i3.1: <i>quantidade</i> é menor ou igual a zero	Erro de incremento com quantidade não positiva
	f6	v1.1: <i>idCompra</i> é igual ao atributo <i>número</i> de uma compra existente cujo atributo <i>fechada</i> é	v2.2: <i>isbn</i> é igual ao atributo <i>isbn</i> de um livro que não consta na compra	i3.1: <i>quantidade</i> é menor ou igual a zero	Erro de adição com quantidade não positiva

		falso			
	f7	v1.1: <i>idCompra</i> é igual ao atributo <i>número</i> de uma compra existente cujo atributo <i>fechada</i> é falso	v2.1: <i>isbn</i> é igual ao atributo <i>isbn</i> de um livro que consta na compra	i3.2: <i>quantidade</i> é maior do que o atributo <i>estoque</i> do livro	Erro de incremento acima do estoque
	f8	v1.1: <i>idCompra</i> é igual ao atributo <i>número</i> de uma compra existente cujo atributo <i>fechada</i> é falso	v2.2: <i>isbn</i> é igual ao atributo <i>isbn</i> de um livro que não consta na compra	i3.2: <i>quantidade</i> é maior do que o atributo <i>estoque</i> do livro	Erro de adição acima do estoque

Dependendo da linguagem de programação que se use, caso ela permita tipagem forte estática e definição de novos tipos primitivos, é possível reduzir a quantidade de testes necessários. No exemplo acima, o parâmetro *quantidade* é assumido como sendo uma variável do tipo *inteiro* e que, portanto, pode ser positivo ou negativo. Assim, como apenas valores positivos podem ser aceitos pela função, foi necessário definir uma classe de equivalência inválida com os números não positivos.

Porém, se a linguagem permitir a criação de novos tipos primitivos fortes, o programador, poderia definir que a variável *quantidade* deve pertencer ao tipo *Natural* que seria definido como o conjunto dos números positivos (excluindo o zero). Dessa forma, nunca aconteceria de esta função receber um valor de *quantidade* igual a zero ou negativo e assim, a classe de equivalência i3.1 deixaria de existir e menos testes teriam que ser feitos.

O leitor poderia perguntar se não seria possível também definir um tipo *ISBNCadastrado* que conteria apenas os valores de *isbn* de livros que constam no cadastro. O problema com isso é que tipos primitivos são estáticos e, por isso, devem ser definidos de forma puramente sintática, ou seja, deve ser possível determinar se um valor pertence ao tipo apenas observando sua estrutura e eventualmente aplicando alguma regra ou fórmula em tempo de *compilação*. Assim, não é possível definir tipos primitivos estáticos em função de um conjunto de dados dinâmicos, como no caso de dados usualmente armazenados em banco de dados, pois isso só poderia ser verificado em tempo de *execução*.

Assim, apenas se for possível determinar que um valor pertence ao tipo sem que seja necessário consultar dados dinâmicos armazenados, ele pode ser definido como tipo primitivo. Exemplos são:

- Números naturais
- Números pares
- Números primos
- Datas (dia, mês e ano)
- Valor de ISBN (apenas a formatação, sem levar em conta se o valor foi atribuído a um livro ou não)
- CPF (apenas formatação)
- Intervalo (duas datas, sendo a primeira necessariamente menor ou igual do que a segunda)

Assim, quando o domínio de um parâmetro é reduzido com o uso de tipos primitivos, algumas classes de equivalência inválidas para esse parâmetro deixam de existir. No melhor dos casos, o tipo primitivo vai conter unicamente valores válidos e assim, para esse parâmetro, não haverá nenhuma classe inválida. Por outro lado, tipos cuja determinação necessita consultar algum conjunto de dados dinâmicos armazenados, não podem ser definidos como tipos primitivos. Exemplos são:

- Os números das compras já efetuadas.
- Os CPFs de pessoas cadastradas.
- As datas que correspondem ao aniversário de uma pessoa cadastrada.

Esse tipo de situação não pode ser resolvido com tipagem. Assim, se a função exige, por exemplo, uma data que corresponde ao aniversário de uma pessoa cadastrada, a tipagem estática só pode garantir que o parâmetro vai conter uma data válida, mas não que ela corresponde a um aniversário. Assim, neste caso, necessariamente deverá existir uma classe de equivalência inválida para datas que não correspondem a este conjunto e ela deverá ser testada.

13.5.2 ANÁLISE DE VALOR LIMÍTROFE

Um dos ditados em teste de software diz que os *bugs* (tradução literal: "insetos") costumam se esconder nas *frestas*. Em função dessa realidade, a técnica de partição de equivalência normalmente é usada em conjunto com o critério de análise de valor limítrofe.

A *análise de valor limítrofe* consiste em considerar não apenas um valor qualquer para teste dentro de uma classe de equivalência, mas um ou mais valores fronteiros com outras classes de equivalência quando isso puder ser determinado.

Em domínios ordenados (números inteiros, por exemplo), esse critério pode ser aplicado. Por exemplo, se um programa exige uma entrada que, para ser válida, deve estar no intervalo $[n..m]$, então existem três classes de equivalência:

- Inválida para qualquer $x < n$.
- Válida para qualquer $n \leq x \leq m$.
- Inválida para qualquer $x > m$.

A análise de valor limítrofe sugere que possíveis erros de lógica do programa não vão ocorrer em pontos arbitrários dentro desses intervalos, mas nos pontos em que um intervalo se encontra com outro. Isso acontece, por exemplo, quando um programador usa, em alguma parte do programa que envolva repetição ou seleção, ">" em vez de ">=", ou quando ele usa n em vez de $n-1$. Assim, no caso do exemplo acima, os seguintes valores limítrofes devem ser testados:

- Para a primeira classe inválida, deve-se testar o valor $n - 1$.
- Para a classe válida, devem-se testar os valores n e m .
- Para a segunda classe inválida, deve-se testar o valor $m + 1$.

Assim, se existir um erro no programa para alguma dessas classes, é muito mais provável que ele seja capturado dessa forma do que se fosse selecionado um valor qualquer dentro de cada um desses três intervalos.

Mas a técnica não diz que *apenas* estes valores devem ser testados. Ela diz que eles devem ser *obrigatoriamente* testados, mas outros valores dentro das classes de equivalência podem também ser testados. Considere, por exemplo, um parâmetro *idade*, do tipo *inteiro* definido de acordo com as seguintes classes de equivalência:

- Menor que 0: inválido.
- Entre 0 e 17: válido – menor de idade.
- Entre 18 e 64: válido – adulto.
- Entre 65 e 120: válido – idoso.
- Acima de 121: inválido – idoso improvável.

Aplicando a técnica de análise de valor limítrofe e escolhendo mais um elemento aleatoriamente dentro de cada intervalo das classes de equivalência, poderiam ser testados, por exemplo, os seguintes valores: -10, -1, 0, 8, 17, 18, 30, 64, 65, 80, 120, 121, 156. Os valores em negrito na lista

anterior correspondem aos valores obrigatórios pela análise de valor limítrofe. Os demais foram aleatoriamente selecionados dentro das respectivas classes de equivalência.

13.5.3 Teste Funcional em Nível de Sistema

Como já foi visto, o teste de sistema, consiste em testar a interface do sistema como se fosse um usuário tentando realizar os fluxos de atividades requeridos. Possivelmente uma das melhores formas de documentar estes fluxos atualmente, sejam os *casos de uso* (Wazlawick, 2015). O teste funcional então, é adequado para este nível de teste porque ele será feito sem que o código em si seja conhecido. A ideia é executar todos os fluxos possíveis do caso de uso e obter os resultados desejados em cada situação.

Um caso de uso possui sempre um *fluxo principal*, também chamado de *caminho feliz*, que é a sequência de ações do usuário e respostas do sistema onde nada dá errado. Por exemplo, ir a um caixa automático, se identificar, solicitar uma quantia em dinheiro, receber essa quantia e ir embora.

Mas existem fluxos alternativos que são definidos para tratar situações de exceção como por exemplo, quando o usuário erra a senha, solicita uma quantia além do seu saldo ou além do saldo da máquina ou fora do horário de operação da máquina. Estes são chamados de *fluxos de exceção* e normalmente são sequências de ações que iniciam em algum ponto do fluxo principal e possivelmente voltam a ele depois. Há ainda as *variantes*, que são fluxos alternativos que não correspondem necessariamente a situações de erro, mas simplesmente a variações do comportamento do usuário. Ele poderia, por exemplo, se identificar usando biometria ao invés de senha, ou poderia, ao invés de enviar o produto ao endereço cadastrado, inserir um novo endereço para envio. Esses fluxos alternativos também sairiam em algum ponto do fluxo principal e possivelmente retornariam a ele depois.

Considere, a título de exemplo, um teste de sistema a ser conduzido para avaliar o seguinte caso de uso simplificado:

Caso de Uso: Comprar livros

Fluxo principal:

1. O usuário insere palavras-chave para pesquisa de livros.
2. O sistema apresenta a lista de livros (título, autor, preço) que atende às palavras-chave.
3. O usuário seleciona um livro e indica a quantidade desejada.
4. O sistema informa a relação de livros no pedido e o valor total do pedido.
5. O usuário finaliza a compra.
6. O sistema confirma a compra.

Fluxos de tratamento de exceção:

E3a: O usuário indica uma quantidade acima do estoque.

E3a.1. O sistema informa a quantidade em estoque do livro em questão.

E3a.2. O usuário informa uma quantidade menor ou igual ao estoque.

Avança ao passo 4 do fluxo principal.

E5a: O usuário ainda não fez login.

E5a.1. O usuário faz login informando nome de usuário e senha.

E5a.2. O sistema confirma o login.

Retorna ao passo 5 do fluxo principal.

Fluxos de variantes:

V5a: O usuário deseja selecionar mais livros na mesma lista.

Retorna ao passo 3 do fluxo principal.
V5b: O usuário deseja fazer uma nova pesquisa.
Retorna ao passo 1 do fluxo principal.

Diferentemente do que acontece com o teste funcional em nível de unidade, onde temos testes de sucesso e de falha, no caso de testes de sistema, o objetivo é sempre chegar ao final do caso de uso passando apenas pelo fluxo principal, ou iniciando no fluxo principal, passando por um fluxo de exceção ou variante e retornando ao fluxo principal. Assim, não há, usualmente, testes de falha, mas testes que avaliam como o sistema se comporta quando ocorrem exceções e variantes e como ele se recupera no caso das exceções.

Como o exemplo acima tem um fluxo principal, duas variantes e duas exceções, então pelo menos cinco testes terão que ser feitos. Estes testes estão resumidos na **Tabela 13.6**.

Tabela 13.6 Casos de teste de sistema para o caso de uso "Comprar livros"

#	Objetivo	Caminho	Como testar	Resultados
1	Fluxo principal	1, 2, 3, 4, 5, 6	Um usuário logado entra com palavras-chave, seleciona um livro, indica uma quantidade dentro do estoque e finaliza a compra	Uma compra com o livro e quantidade indicados foi registrada
2	E3a: O usuário indica uma quantidade acima do estoque.	1, 2, 3, E3a.1, E3a.2, 4, 5, 6	Um usuário logado entra com palavras-chave, seleciona um livro, indica uma quantidade acima do estoque, é informado da quantidade válida, em seguida indica uma quantidade dentro do estoque e finaliza a compra	Uma compra com o livro e quantidade indicados após a informação sobre estoque foi registrada
3	E5a: O usuário ainda não fez login.	1, 2, 3, 4, 5, E5a.1, E5a.2, 6	Um usuário não logado entra com palavras-chave, seleciona um livro, indica uma quantidade dentro do estoque e finaliza a compra. É então solicitado a fazer login, faz login e finaliza a compra novamente	Uma compra com o livro e quantidade indicados foi registrada
4	V5a: O usuário deseja selecionar mais livros na mesma lista.	1, 2, 3, 4, 5, 3, 4, 5, 6	Um usuário logado entra com palavras-chave, seleciona um livro, indica uma quantidade dentro do estoque, em seguida seleciona outro livro e indica uma quantidade dentro do estoque e finaliza a compra	Uma compra com os livros e respectivas quantidades foi registrada
5	V5b: O usuário deseja fazer uma nova pesquisa.	1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 6	Um usuário logado entra com palavras-chave, seleciona um livro, indica uma quantidade dentro do estoque, em seguida faz uma nova pesquisa, seleciona outro livro e indica uma quantidade dentro do estoque e finaliza a compra	Uma compra com os livros e respectivas quantidades foi registrada

Recomenda-se que estes testes de sistema sejam feitos em um primeiro momento por um testador humano, mas que depois sejam automatizados. É importante uma primeira visão humana porque por mais sofisticados que sejam os robôs de automatização de teste, há coisas que por enquanto só o olho humano é capaz de avaliar, coisas como botões ou campos mal distribuídos, cores desagradáveis, lógica de navegação não alinhada com as necessidades dos usuários etc. Há também a possibilidade de o testador humano ter alguma intuição de testes não previstos no plano original, que podem ser adicionados à sequência de casos de teste.

Depois de passar, então por esta fase, o roteiro com os casos de teste pode (e deve) ser repassado a uma equipe especializada em programação de teste automatizado de sistema. Ferramentas de código aberto, como *Selenium* (ver *QR code*), são bastante populares para realizar este tipo de automatização. [QRC 13.1]



13.6 TDD – Desenvolvimento Orientado por Testes

TDD (*Test Driven Development*) ou *Desenvolvimento Dirigido por Testes* (Beck, 2003) é uma técnica ou filosofia de programação que incorpora o teste ao processo de produção de código da seguinte forma:

1. Primeiramente o desenvolvedor, que recebe a especificação de uma nova funcionalidade a ser implementada, deve programar um conjunto de testes automatizados para testar o código que ainda não existe.
2. Esse conjunto de testes deve ser executado e falhar. Isso é feito para mostrar que os testes efetivamente têm algum poder de verificação e não terão sucesso a não ser que o código específico seja desenvolvido. Se o código passar em algum desses testes, é possível que ou o teste não seja efetivo, ou que a característica a ser implementada já exista no sistema.
3. Em seguida, o código deve ser desenvolvido da forma mais minimalista possível, isto é, apenas para passar nos testes.
4. Depois que o código passar em todos os testes, ele pode ser refatorado, para atender a padrões de qualidade interna, e testado novamente até que passe em todos os testes novamente.

A Figura 13.11 apresenta o diagrama de atividades para esse tipo de processo.

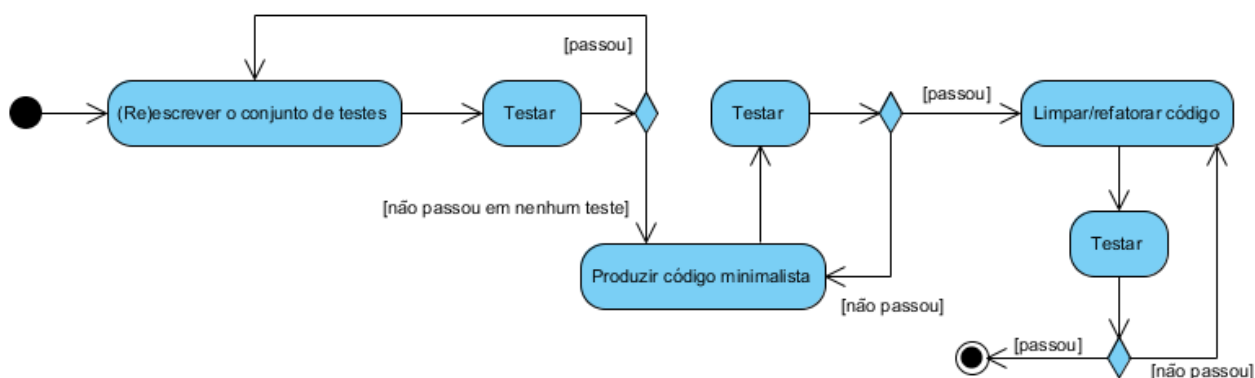


Figura 13.11 Diagrama de atividades para realização de TDD

Um dos pontos-chave nesse processo é que, depois que os testes foram estabelecidos, o programador não deve implementar nenhuma funcionalidade além daquelas para as quais os testes foram criados. Isso evita que os programadores percam tempo criando estruturas que efetivamente não

eram necessárias desde o início. Isso satisfaz ao princípio KISS (*Keep it Simple, Stupid!*)¹

¹“Mantenha simples, tolo!”.

, que indica que seria tolice fazer algo mais do que o mais simples necessário em um projeto, e também o princípio YAGNI (*You Ain't Gonna Need It*)²

²“Você não vai precisar disso”.

Para escrever um conjunto de testes efetivo, é necessário que o programador primeiro compreenda perfeitamente os requisitos e a especificação do componente que vai desenvolver. Nesse sentido, os contratos de operação de sistema (pré e pós condições) e o roteiro de teste funcional correspondente podem ajudar muito.

A técnica é bastante exigente em relação ao processo a ser seguido. Por exemplo, um programador que descubra a necessidade de adicionar um ELSE a um IF em um código já aprovado nos testes deverá, antes de escrever o código, escrever um teste para o ELSE, certificar-se de que o teste falha para o programa atual e somente depois adicionar o ELSE ao código. Pular etapas não é encorajado pela técnica.

Como exemplo, considere que um programador recebeu a incumbência de desenvolver uma função que recebe um valor inteiro, que corresponde à idade de uma pessoa. Esse programa deve retornar a string "menor", caso a idade esteja entre 0 e 17 e "adulto" caso a idade esteja acima de 17. Nada foi especificado sobre idades acima de 100 anos, mas idades negativas devem ser rejeitadas, ou seja, o programa deve parar a execução indicado uma exceção se ele receber uma idade negativa. O parâmetro *idade*, assim, teria três classes de equivalência:

- *Menor*, classe válida de 0 a 17 anos.
- *Adulto*, classe válida a partir de 18 anos.
- *Não nascido*, classe inválida, até -1 anos.

A partir daí, usando TDD, o programador deve primeiramente desenvolver o código que vai testar essas condições pela chamada da função *categoria(idade)*, que ainda não foi implementada. Possivelmente o programador poderia desenvolver em um primeiro momento um código como este:

```
01 def categoria(idade):
02     return
03
04 #teste de classes válidas usando valor limítrofe
05 menor = 'menor'
06 adulto = 'adulto'
07 assert categoria(0) == menor
08 assert categoria(8) == menor
09 assert categoria(17) == menor
10 assert categoria(18) == adulto
11 assert categoria(45) == adulto
12
13
14 #teste de classes inválidas usando valor limítrofe
15 for idade in [-20, -1]:
16     try:
17         categoria(idade)
18         raise Exception
19     except: pass
```

Observe que a função declarada nas linhas 1 e 2, nada retorna (na prática, em Python, ela acaba retornando o objeto *None*). Quando esse programa for executado, ele vai parar com erro na linha

7, que é exatamente o resultado que se espera, já que estão sendo rodados testes em um programa que ainda não foi construído. Qualquer outro resultado (parar na linha 8 ou 9, por exemplo), indicaria que há algo errado no programa ou nos testes.

Em relação à escolha dos testes, em função das classes de equivalência e seus valores limítrofes, são testados os valores 0, 8 e 17 da classe "menor" (sendo, 0 e 17 os valores limítrofe), e 18 e 45 para a classe "adulto" (sendo 18 o único valor limítrofe, já que não foi estabelecido um limite superior para esta classe).

Já para a classe inválida de valores negativos, são testados -20 e -1 (sendo -1 o valor limítrofe). Observe que o teste, neste caso, deve confirmar que, caso a função receba um valor negativo, ela gere uma exceção. Assim, o programa entre as linhas 14 e 19 tenta executar a função com cada um dos valores inválidos e, se for gerada uma exceção, o programa pula para a linha 19, que aceita o resultado e segue executando o comando *pass*. Porém, se a função *categoria* não gerar a exceção esperada, ela vai retornar algum valor espúrio e assim, seguir executando pela linha 18, a qual, por sua vez, vai parar o programa gerando uma exceção e falhando, portanto, o teste.

Uma observação é que aqui nestes exemplos se usam apenas as facilidades de teste já embutidas no núcleo da linguagem Python. Existem *frameworks* de teste que permitem obter resultados mais sofisticados, como, por exemplo, mostrar o resultado de todos os testes e não apenas até o programa parar, ou ainda, permitir visualizar na Web, em tempo real, o status dos testes sendo realizados.

O próximo passo consiste em o programador implementar um código minimalista que apenas passe nos testes, sem fazer quaisquer outras coisas não definidas. Esse programa poderia então ser construído assim:

```
01 def categoria(idade):
02     assert idade >= 0
03     if idade <= 18:
04         return "menor"
05     else:
06         return "adulto"
07     return
08
09 #teste de classes válidas usando valor limítrofe
10 menor = 'menor'
11 adulto = 'adulto'
12 assert categoria(0) == menor
13 assert categoria(8) == menor
14 assert categoria(17) == menor
15 assert categoria(18) == adulto
16 assert categoria(45) == adulto
17
18 #teste de classes inválidas usando valor limítrofe
19 for idade in [-20, -1]:
20     try:
21         categoria(idade)
22         raise Exception
23     except: pass
```

Feito isso, o programador pode executar os testes novamente. Ele vai certamente perceber que o programa vai parar com erro na linha 15, indicando que a função não passou no teste que verifica a idade 18. Analisando melhor o código, o programador poderá perceber que se equivocou na linha

3 por usar o sinal "<=" em vez de "<". Esse exemplo demonstra porque a técnica de análise de valor limítrofe é tão importante, já que o programa teria passado nos testes para qualquer outro valor maior do que 18.

Feita a correção na linha 3, o programa vai passar em todos os testes. Se necessário, neste momento, poderia ser feita alguma refatoração ou limpeza de código e, depois de passar novamente nos testes, o programa seguiria para integração ao código do sistema ao qual ele vai servir.

Agora, imagine que dentro de algumas semanas, esse programador ou um outro programador qualquer, receba a incumbência de alterar a função *categoria* de forma que ela também verifique se a idade é igual ou superior a 65 anos. Neste caso, deve ser retornada a string "idoso". Além disso, se a idade for superior a 120 anos deve ser gerada uma exceção. Assim, o parâmetro *idade* da função *categoria* passa a ter as seguintes classes de equivalência:

- *Menor*, classe válida de 0 a 17 anos.
- *Adulto*, classe válida de 18 a 64 anos.
- *Idoso*, classe válida de 65 a 120 anos.
- *Não nascido*, classe inválida, até -1 anos.
- *Improvável*, classe inválida, a partir de 121 anos.

Assim, de acordo com a técnica, *antes de fazer qualquer modificação no código da função "categoria"*, o programador deve refazer o conjunto de testes, para que reflitam a nova especificação da função. O resultado será este:

```
01 def categoria(idade):
02     assert idade >= 0
03     if idade < 18:
04         return "menor"
05     else:
06         return "adulto"
07     return
08
09 #teste de classes válidas usando valor limítrofe
10 menor = 'menor'
11 adulto = 'adulto'
12 idoso = 'idoso'
13 assert categoria(0) == menor
14 assert categoria(8) == menor
15 assert categoria(17) == menor
16 assert categoria(18) == adulto
17 assert categoria(45) == adulto
18 assert categoria(64) == adulto
19 assert categoria(65) == idoso
20 assert categoria(80) == idoso
21 assert categoria(120) == idoso
22
23 #teste de classes inválidas usando valor limítrofe
24 for idade in [-20, -1, 121, 200]:
25     try:
26         categoria(idade)
27         raise Exception
28     except: pass
```

Agora que os testes foram atualizados para refletir os novos requisitos, o programador deve rodá-los com a função *categoria* inalterada. A ideia é que a função não deve passar nos testes, já que os

requisitos mudaram. Se por acaso a função passar nos testes é porque ela passou por coincidência, o que pode indicar que testes insuficientes foram planejados, ou ainda, que a funcionalidade considerada nova na verdade já era contemplada pela função.

Porém, no caso deste exemplo, isso não vai acontecer, pois quando o programa for novamente testado, ele vai parar com erro na linha 19, indicando que não foi encontrado o resultado correto para *categoria*(65), que deveria ser "idoso".

A partir daí o programador vai fazer os ajustes necessários no programa para que ele passe nesses novos testes, mas nada mais do que isso. O resultado poderá ser este:

```
01 def categoria(idade):
02     assert idade >= 0
03     assert idade <= 120
04     if idade < 18:
05         return "menor"
06     elif idade < 65:
07         return "adulto"
08     else:
09         return "idoso"
10     return
11
12 #teste de classes válidas usando valor limítrofe
13 menor = 'menor'
14 adulto = 'adulto'
15 idoso = 'idoso'
16 assert categoria(0) == menor
17 assert categoria(8) == menor
18 assert categoria(17) == menor
19 assert categoria(18) == adulto
20 assert categoria(45) == adulto
21 assert categoria(64) == adulto
22 assert categoria(65) == idoso
23 assert categoria(80) == idoso
24 assert categoria(120) == idoso
25
26 #teste de classes inválidas usando valor limítrofe
27 for idade in [-20, -1, 121, 200]:
28     try:
29         categoria(idade)
30         raise Exception
31     except: pass
```

Assim, quando o programa for executado, ele vai passar em todos os testes. Em seguida, ele pode ser novamente, refatorado, se necessário, testado pela última vez e encaminhado para integração.

Um desenvolvimento extra de TDD pode ser atingido quando se consegue automatizar os testes de aceitação do usuário. Essa abordagem, conhecida com ATDD (*Acceptance Test-Driven Development*; Koskela, 2007), disponibiliza aos desenvolvedores a garantia de que os requisitos especificados estão sendo atendidos, mesmo que o usuário não esteja presente todo o tempo.

13.7 Medição em Teste

Uma informação importante a ser relatada por equipes de teste é o *status* da atividade de teste (Crowther, 2012), o que deve ser feito de forma precisa e compreensiva. Então, nesse caso, aplica-se também o conceito de métrica para avaliação objetiva de *status*. Pode-se falar em dois grandes

conjuntos de métricas de teste:

- *Métricas do processo de teste*: as medidas relacionadas reportam a quantidade de testes requeridos, planejados, executados etc., mas não o estado do produto em si.
- *Métricas de teste do produto*: as medidas relacionadas reportam o estado do produto em relação à atividade de teste, por exemplo, quantos defeitos foram encontrados, quantos estão em revisão, quantos foram resolvidos etc.

As seguintes medições relacionadas às métricas de processo de teste poderiam ser apresentadas durante a preparação e a execução dos testes:

- Número de testes previstos (sua necessidade já foi identificada).
- Número de testes planejados (casos de teste definidos).
- Número de testes executados, incluindo: (a) testes que falharam e (b) testes que foram aprovados.

Essas métricas podem ser comparadas ou relativizadas com outras métricas usualmente tomadas sobre o processo, como o tempo investido nas atividades, o número de pontos de função etc.

Alguns exemplos de métricas de teste relativas ao produto são:

- Número de defeitos descobertos.
- Número de defeitos corrigidos.
- Distribuição dos defeitos por gravidade.
- Distribuição dos defeitos por classe ou pacote.

As medições relacionadas a essas métricas podem ser apresentadas em gráficos que permitem visualizar o desempenho das equipes de desenvolvimento e testes, em que se espera que o número de defeitos descobertos seja relativamente constante no tempo e que as atividades de correção atinjam seus objetivos também de forma constante, até que o produto esteja suficientemente livre de defeitos para ser implantado (**Figura 13.12**).

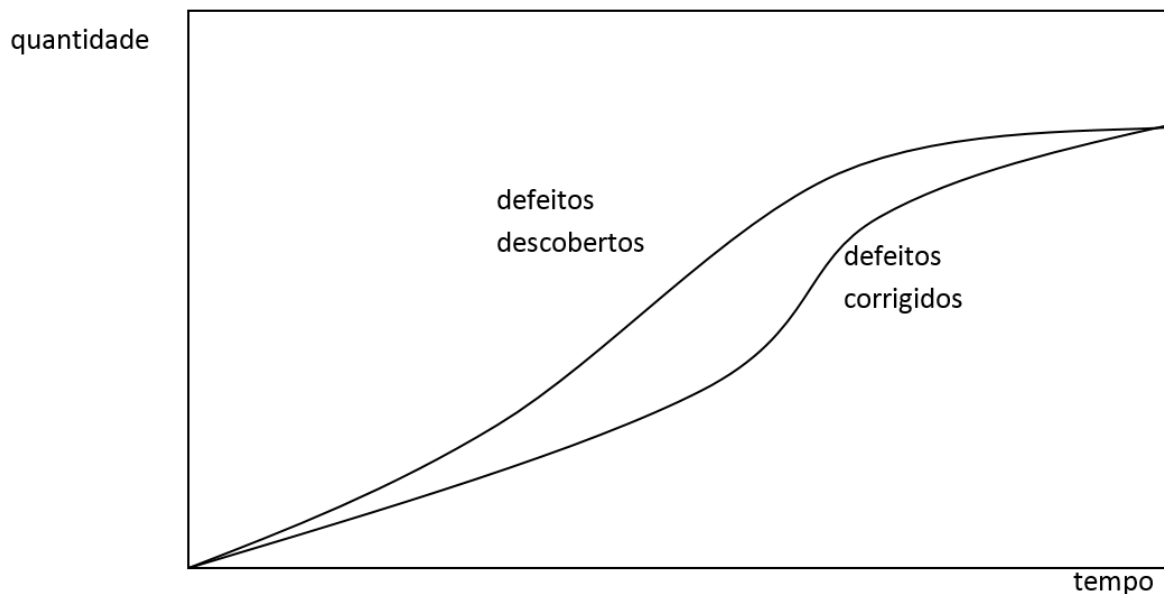


Figura 13.12 Evolução esperada das medidas de defeitos de produto ao longo de um projeto

Outras medidas relacionadas a teste são o tempo gasto com as atividades de teste (isto é, o custo

dos testes) e a cobertura dos testes, isto é, quantas funcionalidades do sistema tiveram seus testes efetivamente planejados, executados e aprovados.

13.8 Depuração

Depuração é o processo de remover defeitos de um programa. Normalmente se inicia com atividades de teste, que identificam o problema, ou a partir de inspeções de código, ou ainda a partir de relatos de usuários. No último caso, a equipe deve, no início do processo de depuração, tentar reproduzir o defeito relatado pelo usuário, o que nem sempre é possível ou fácil.

A depuração é uma atividade artesanal em que a quantidade e a variedade de casos dificultam a elaboração de um processo padrão. Porém, a adoção de boas técnicas de engenharia de software, como integrações frequentes, controle de versões, desenvolvimento orientado a testes etc., podem facilitar bastante o processo de descoberta e correção de defeitos.

A tarefa de depuração pode ser tão simples quanto corrigir uma *string* com erro ortográfico ou tão complexa a ponto de exigir um trabalho de pesquisa, coleta de dados e formulação de hipóteses digna dos melhores detetives.

De forma geral, ferramentas, conhecidas como *debuggers*, ajudam no trabalho de depuração, pois permitem executar programas passo a passo, observando o valor das variáveis, parar, retornar etc.

REMISSIVO DO CAPÍTULO

- análise de valor limítrofe, 32, 37, 38, 43
- autenticação, 11
- autorização, 11
- beta, 8, 9
- business process modeling and notation*, 8
- caminhos independentes, 21, 22, 24, 25, 26, 30
- CASE, 16, 17
- caso de uso, 7, 10, 38, 39, 40
- caso de teste, 1, 2, 4, 12, 20, 22, 23, 24, 26, 30, 31, 35, 36, 40, 45
- classes de equivalência, 32, 35, 37, 38, 42, 43
- classe
 - inválida, 33, 34, 35, 42, 43, 44, 45
 - válida, 33, 34, 35, 42, 43, 44, 45
- complexidade ciclomática, 13, 14, 18, 19, 20, 21, 24, 25, 28, 30
- confidencialidade, 11
- debuggers*, 47
- defeito, 1, 2, 3, 9, 23, 31, 46
- depuração, 1, 3, 46, 47
- desenvolvimento dirigido por testes, 1, 6, 31, 40, 41, 42, 45
- disponibilidade, 11
- driver*, 4, 5, 6
- engano, 2
- erro, 1, 2, 3, 10, 35, 38, 39, 42, 43, 44, 47
- falha, 1, 2, 12, 23, 32, 35, 39, 41
- Fermat, 28, 29

- fixture*, 5, 6
- grafo de fluxo, 14, 18, 19, 20, 21, 24, 27, 29
- homologação, 3, 8
- integração
 - big-bang*, 6
 - bottom-up*, 6
 - incremental, 7
 - sanduíche, 6
 - top-down*, 6
- integridade de informação, 11
- KISS, 41
- Lei de Murphy, 1
- manutenção adaptativa, 9
- não repúdio, 11
- particionamento de equivalência, 33
- pós condições, 33
- precondições, 33
- projeto, 46
- stub, 3, 4, 5, 7
- TDD. Consulte desenvolvimento dirigido por testes
- test driven development*. Consulte desenvolvimento dirigido por testes
- teste, 39
 - de aceitação, 8
 - de carga, 10, 11
 - de ciclo de negócio, 8
 - de estresse, 11
 - de instalação, 12
 - de integração, 6
 - de interface com usuário, 10
 - de operação, 9
 - de performance, 10
 - de regressão, 9
 - de resistência, 11
 - de robustez, 12
 - de sistema, 7, 8, 9, 10, 38, 39, 40
 - de unidade, 6, 32
 - estrutural, 1, 6, 13, 29, 31, 32
 - funcional, 1, 5, 6, 31, 32, 38, 39, 41
 - de funcionalidade, 5, 9
 - de segurança, 11
 - de unidade, 5, 6, 7, 12
 - suplementar, 1, 5, 9
- throw-away*, 5
- tipo primitivo, 37
- tolerância a falhas, 2, 5, 9
- validação, 3, 8

verificação, 3, 8, 31, 41
YAGNI, 41